

Rent price prediction in Berlin

Model training and picking the most suitable model

```
In [8]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import altair as alt
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.compose import TransformedTargetRegressor
from IPython.display import display, HTML
from IPython.display import Markdown as md
```

```
In [9]: import sys
import os

sys.path.append(os.path.abspath("../models"))

from rent_pipeline import (
    load_data,
    split_data,
    make_cv,
    evaluate_model,
    save_bundle,
    load_bundle,
    predict_listing,
    build_preprocessor,
    build_pipeline
)
```

```
In [10]: df = load_data('../datasets/rent_listings_final.csv')
df.shape
```

Out[10]: (15588, 23)

```
In [11]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15588 entries, 0 to 15587
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   listing_url            15588 non-null  object
1   cold_rent              15588 non-null  float64
2   area                  15588 non-null  float64
3   rooms                 15588 non-null  float64
4   floor                 13646 non-null  float64
5   free_from             15588 non-null  object
6   has_balkon            15588 non-null  bool
7   has_terrasse          15588 non-null  bool
8   has_garten            15588 non-null  bool
9   elevator              15588 non-null  bool
10  parking               15588 non-null  bool
11  has_basement          15588 non-null  bool
12  is_barrier-free       15588 non-null  bool
13  has_built-in_kitchen  15588 non-null  bool
14  has_bathtub           15588 non-null  bool
15  has_shower            15588 non-null  bool
16  flooring_type         15588 non-null  object
17  energy_source         15588 non-null  object
18  heating_type          15588 non-null  object
19  property_condition    15588 non-null  object
20  year_built            7662 non-null   float64
21  zip_code              15588 non-null  int64
22  district              15588 non-null  object
dtypes: bool(10), float64(5), int64(1), object(7)
memory usage: 1.7+ MB
```

```
In [12]: df = df.drop(columns=["listing_url", "zip_code"])
```

```
In [13]: # So we get reproducible results:
np.random.seed(16)
```

Part 1: Modeling and model evaluation

1. Split the dataset into a train set (80%) and a test set (20%).

```
In [14]: X_train, X_test, y_train, y_test = split_data(df, test_size=0.2, random_state=16)
```

2. Modeling and training

```
In [15]: from sklearn.linear_model import LinearRegression

metrics_linear, bestmodel_linear = evaluate_model(
    "linear_regression",
    LinearRegression(),
    {},
    X_train,
    y_train,
    X_test,
    y_test
)

metrics_linear
```

```
Out[15]: {'model': 'linear_regression',
          'best_params': {},
          'cv_rmse': 432.648392221836,
          'train_rmse': 428.58787681245644,
          'test_rmse': 408.98770317538276,
          'train_r2': 0.6663174481981542,
          'test_r2': 0.6796527825087897,
          'test_mae': 289.3131410640148,
          'overfit_gap_r2': -0.013335334310635494}
```

```
In [16]: display(md(f'<span style="font-weight:500">Linear regression achieved an R2 score of {metrics_linear["test_r2"]:.4f}, which is not ve
```

Linear regression achieved an R^2 score of 0.6797, which is not very good. However, this is expected because a simple linear model cannot capture the complex nonlinear relationships present in rental prices.

```
In [17]: from sklearn.neighbors import KNeighborsRegressor

metrics_knn, bestmodel_knn = evaluate_model(
    "knn",
    KNeighborsRegressor(algorithm="ball_tree"),
    {
        "model__n_neighbors": [3, 5],
        "model__weights": ["distance"],
    },
    X_train,
    y_train,
    X_test,
    y_test
)

metrics_knn
```

```
Out[17]: {'model': 'knn',
          'best_params': {'model__n_neighbors': 3, 'model__weights': 'distance'},
          'cv_rmse': 295.7847761100729,
          'train_rmse': 46.105525357852365,
          'test_rmse': 265.5247932558738,
          'train_r2': 0.9961384675957481,
          'test_r2': 0.8649761105915468,
          'test_mae': 153.74112245098948,
          'overfit_gap_r2': 0.1311623570042013}
```

```
In [18]: display(md(f'<span style="font-weight:500">KNN performs slightly better, with an R2 score of {metrics_knn["test_r2"]:.4f} , but not g
```

KNN performs slightly better, with an R^2 score of 0.8650, but not good enough to be considered our final model.

```
In [19]: from sklearn.svm import SVR

metrics_svr, bestmodel_svr = evaluate_model(
    "SVR",
    TransformedTargetRegressor(
        regressor=SVR(),
        transformer=StandardScaler()
    ),
    {
        "model__regressor__kernel": ["linear"],
        "model__regressor__C": [0.1, 1, 10, 100],
        "model__regressor__epsilon": [0.01, 0.05, 0.1, 0.2, 0.5]
    },
    X_train,
    y_train,
    X_test,
    y_test
)
```

```
)  
metrics_svr
```

```
Out[19]: {'model': 'SVR',  
         'best_params': {'model__regressor__C': 0.1,  
                        'model__regressor__epsilon': 0.5,  
                        'model__regressor__kernel': 'linear'},  
         'cv_rmse': 437.011182138511,  
         'train_rmse': 434.31287558571347,  
         'test_rmse': 408.3316662743395,  
         'train_r2': 0.657343367322307,  
         'test_r2': 0.6806796644462924,  
         'test_mae': 286.55976382177033,  
         'overfit_gap_r2': -0.023336297123985328}
```

SVR performs very poorly in this dataset

```
In [20]: from sklearn.ensemble import RandomForestRegressor  
  
metrics_rf, bestmodel_rf = evaluate_model(  
    "Random Forest",  
    RandomForestRegressor(random_state=42),  
    {  
        "model__n_estimators": [100, 200, 300, 400],  
        "model__max_depth": [None, 10, 20, 30, 40],  
        "model__min_samples_split": [2, 5, 10, 20],  
        "model__min_samples_leaf": [1, 2, 4, 8],  
        "model__max_features": ["sqrt", "log2"],  
    },  
    X_train,  
    y_train,  
    X_test,  
    y_test  
)  
  
metrics_rf
```

```
Out[20]: {'model': 'Random Forest',  
         'best_params': {'model__max_depth': 30,  
                        'model__max_features': 'sqrt',  
                        'model__min_samples_leaf': 1,  
                        'model__min_samples_split': 2,  
                        'model__n_estimators': 200},  
         'cv_rmse': 234.58062413897596,  
         'train_rmse': 108.98730031958362,  
         'test_rmse': 209.19388040533767,  
         'train_r2': 0.9784222814781925,  
         'test_r2': 0.916189504775013,  
         'test_mae': 125.33442477938395,  
         'overfit_gap_r2': 0.062232776703179504}
```

```
In [21]: from IPython.display import Markdown as md  
display(md(f'<span style="font-weight:500">We got a good R squared of {metrics_rf["test_r2"]:.4f} and lower errors. This is the best r
```

We got a good R squared of 0.9162 and lower errors. This is the best model so far. We should aim for a higher R-squared.

```
In [23]: from xgboost import XGBRegressor  
  
param_grid_xgb = {  
    "model__n_estimators": [400, 600],  
    "model__max_depth": [5, 7],  
    "model__learning_rate": [0.001, 0.01, 0.1],  
    "model__subsample": [0.8, 1.0],  
    "model__colsample_bytree": [0.8, 1.0],  
    "model__min_child_weight": [1, 3],  
    "model__gamma": [0, 0.1]  
}  
  
metrics_xgb, bestmodel_xgb = evaluate_model(  
    "XGBoost",  
    XGBRegressor(  
        random_state=42,  
        objective="reg:squarederror"  
    ),  
    param_grid_xgb,  
    X_train,  
    y_train,  
    X_test,  
    y_test  
)  
  
metrics_xgb
```

```
Out[23]: {'model': 'XGBoost',
'best_params': {'model__colsample_bytree': 0.8,
'model__gamma': 0.1,
'model__learning_rate': 0.1,
'model__max_depth': 7,
'model__min_child_weight': 1,
'model__n_estimators': 600,
'model__subsample': 1.0},
'cv_rmse': 213.5257409699028,
'train_rmse': 115.66084088666365,
'test_rmse': 184.13917281244596,
'train_r2': 0.9756988724814845,
'test_r2': 0.9350629087994863,
'test_mae': 111.74026820272724,
'overfit_gap_r2': 0.04063596368199818}
```

Both XGBoost and RandomForest have a really good performance with XGBoost having a slightly higher R-squared of almost 0.935, which, considering our data set, is a very good and realistic performance.

Personally, I thought that XGBoost was going to be the best expected model and most likely this is the model we are going to deploy.

If you can notice, the hyperparameters of the best-performing model are the highest values, so after picking the final model, I would prefer to try it with other values one last time and picking the ones that don't cause overfitting.

```
In [24]: import tensorflow as tf
from tensorflow.keras import layers, models, callbacks

def build_nn(input_dim):

    model = models.Sequential()

    model.add(layers.Input(shape=(input_dim,)))

    model.add(layers.Dense(256, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(0.001)))
    model.add(layers.BatchNormalization())
    model.add(layers.Dropout(0.2))

    model.add(layers.Dense(128, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(0.001)))
    model.add(layers.BatchNormalization())
    model.add(layers.Dropout(0.2))

    model.add(layers.Dense(64, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(0.001)))

    model.add(layers.Dense(32, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(0.001)))

    model.add(layers.Dense(1))

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate = 0.0005),
        loss='mse',
        metrics=[tf.keras.metrics.RootMeanSquaredError()]
    )

    return model
```

```
In [29]: from sklearn.metrics import (
mean_squared_error,
mean_absolute_error,
r2_score,
)

def evaluate_nn(
    X_train,
    y_train,
    X_test,
    y_test,
    epochs=300,
    batch_size=64,
    feature_cols=None
):
    nn_pipeline = Pipeline([
        ("preprocessor", build_preprocessor(feature_cols))
    ])

    X_train_processed = nn_pipeline.fit_transform(X_train)
    X_test_processed = nn_pipeline.transform(X_test)

    nn_model = build_nn(X_train_processed.shape[1])
```

```

early_stop = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=20,
    restore_best_weights=True
)

history = nn_model.fit(
    X_train_processed,
    y_train,
    validation_split=0.2,
    epochs=epochs,
    batch_size=batch_size,
    callbacks=[early_stop],
    verbose=1
)

y_pred_train = nn_model.predict(
    X_train_processed, verbose=0
).ravel()

y_pred_test = nn_model.predict(
    X_test_processed, verbose=0
).ravel()

train_r2 = r2_score(y_train, y_pred_train)
test_r2 = r2_score(y_test, y_pred_test)

metrics = {
    "model": "Neural Network",
    "best_params": {
        "epochs": epochs,
        "batch_size": batch_size
    },
    "cv_rmse": None,
    "train_rmse": float(
        np.sqrt(mean_squared_error(y_train, y_pred_train))
    ),
    "test_rmse": float(
        np.sqrt(mean_squared_error(y_test, y_pred_test))
    ),
    "train_r2": float(train_r2),
    "test_r2": float(test_r2),
    "test_mae": float(
        mean_absolute_error(y_test, y_pred_test)
    ),
    "overfit_gap_r2": float(train_r2 - test_r2),
}

return metrics, nn_model, nn_pipeline, history

```

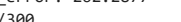
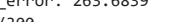
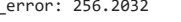
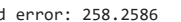
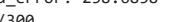
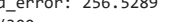
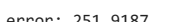
```

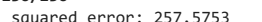
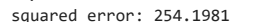
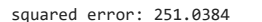
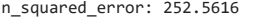
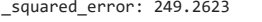
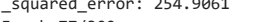
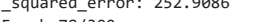
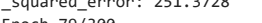
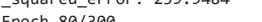
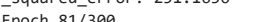
In [30]: metrics_nn, bestmodel_nn, nn_pipeline, history_nn = evaluate_nn(
    X_train,
    y_train,
    X_test,
    y_test,
    epochs=300,
    batch_size=64
)

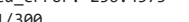
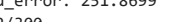
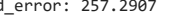
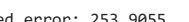
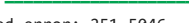
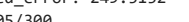
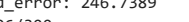
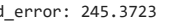
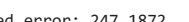
metrics_nn

```

Epoch 1/300
156/156 ————— 9s 13ms/step - loss: 1934628.7500 - root_mean_squared_error: 1390.9092 - val_loss: 1721823.0000 - val_root_mean_squared_error: 1312.1825
Epoch 2/300
156/156 ————— 1s 8ms/step - loss: 685842.2500 - root_mean_squared_error: 828.1556 - val_loss: 323690.6562 - val_root_mean_squared_error: 568.9376
Epoch 3/300
156/156 ————— 2s 10ms/step - loss: 257488.2188 - root_mean_squared_error: 507.4323 - val_loss: 195937.2188 - val_root_mean_squared_error: 442.6471
Epoch 4/300
156/156 ————— 2s 10ms/step - loss: 188379.7812 - root_mean_squared_error: 434.0264 - val_loss: 152753.9062 - val_root_mean_squared_error: 390.8364
Epoch 5/300
156/156 ————— 2s 11ms/step - loss: 165258.9375 - root_mean_squared_error: 406.5195 - val_loss: 124662.1562 - val_root_mean_squared_error: 353.0741
Epoch 6/300
156/156 ————— 1s 9ms/step - loss: 137505.7656 - root_mean_squared_error: 370.8165 - val_loss: 105146.4219 - val_root_mean_squared_error: 324.2616
Epoch 7/300
156/156 ————— 1s 9ms/step - loss: 123827.5312 - root_mean_squared_error: 351.8901 - val_loss: 98293.2891 - val_root_mean_squared_error: 313.5162
Epoch 8/300
156/156 ————— 1s 9ms/step - loss: 112957.6484 - root_mean_squared_error: 336.0903 - val_loss: 92738.8281 - val_root_mean_squared_error: 304.5291
Epoch 9/300
156/156 ————— 1s 9ms/step - loss: 108685.8672 - root_mean_squared_error: 329.6740 - val_loss: 88567.6953 - val_root_mean_squared_error: 297.6017
Epoch 10/300
156/156 ————— 1s 9ms/step - loss: 105590.8594 - root_mean_squared_error: 324.9460 - val_loss: 85379.0625 - val_root_mean_squared_error: 292.1953
Epoch 11/300
156/156 ————— 2s 12ms/step - loss: 98064.3047 - root_mean_squared_error: 313.1507 - val_loss: 83209.8125 - val_root_mean_squared_error: 288.4594
Epoch 12/300
156/156 ————— 3s 18ms/step - loss: 91389.9922 - root_mean_squared_error: 302.3062 - val_loss: 80790.9531 - val_root_mean_squared_error: 284.2358
Epoch 13/300
156/156 ————— 3s 16ms/step - loss: 91742.6875 - root_mean_squared_error: 302.8890 - val_loss: 81155.5312 - val_root_mean_squared_error: 284.8764
Epoch 14/300
156/156 ————— 2s 12ms/step - loss: 90994.7031 - root_mean_squared_error: 301.6516 - val_loss: 76454.5078 - val_root_mean_squared_error: 276.5023
Epoch 15/300
156/156 ————— 1s 9ms/step - loss: 84922.7422 - root_mean_squared_error: 291.4134 - val_loss: 76777.6562 - val_root_mean_squared_error: 277.0860
Epoch 16/300
156/156 ————— 1s 9ms/step - loss: 81068.0703 - root_mean_squared_error: 284.7228 - val_loss: 78508.6641 - val_root_mean_squared_error: 280.1922
Epoch 17/300
156/156 ————— 1s 9ms/step - loss: 82690.9922 - root_mean_squared_error: 287.5587 - val_loss: 75881.5859 - val_root_mean_squared_error: 275.4643
Epoch 18/300
156/156 ————— 1s 9ms/step - loss: 79579.9453 - root_mean_squared_error: 282.0974 - val_loss: 75731.6953 - val_root_mean_squared_error: 275.1921
Epoch 19/300
156/156 ————— 1s 9ms/step - loss: 76580.5469 - root_mean_squared_error: 276.7300 - val_loss: 72829.1484 - val_root_mean_squared_error: 269.8668
Epoch 20/300
156/156 ————— 2s 11ms/step - loss: 75498.5859 - root_mean_squared_error: 274.7682 - val_loss: 74022.2266 - val_root_mean_squared_error: 272.0684
Epoch 21/300
156/156 ————— 2s 11ms/step - loss: 73254.5938 - root_mean_squared_error: 270.6539 - val_loss: 73114.3750 - val_root_mean_squared_error: 270.3948
Epoch 22/300
156/156 ————— 2s 9ms/step - loss: 74348.7578 - root_mean_squared_error: 272.6677 - val_loss: 73809.5703 - val_root_mean_squared_error: 271.6772
Epoch 23/300
156/156 ————— 1s 8ms/step - loss: 71522.5312 - root_mean_squared_error: 267.4350 - val_loss: 72096.0547 - val_root_mean_squared_error: 268.5051
Epoch 24/300
156/156 ————— 2s 10ms/step - loss: 69090.1562 - root_mean_squared_error: 262.8481 - val_loss: 70172.7969 - val_root_mean_squared_error: 264.8995
Epoch 25/300
156/156 ————— 2s 11ms/step - loss: 70045.8672 - root_mean_squared_error: 264.6598 - val_loss: 73161.4609 - val_root_mean_squared_error: 270.4818
Epoch 26/300
156/156 ————— 2s 11ms/step - loss: 69931.6016 - root_mean_squared_error: 264.4438 - val_loss: 69684.8359 - val_root_mean_squared_error: 263.9768
Epoch 27/300
156/156 ————— 2s 11ms/step - loss: 70320.6016 - root_mean_squared_error: 265.1783 - val_loss: 70151.5234 - val_root_mean_squared_error: 264.8593
Epoch 28/300
156/156 ————— 2s 13ms/step - loss: 69803.6094 - root_mean_squared_error: 264.2016 - val_loss: 69221.9609 - val_root_mean_squared_error: 263.0986

Epoch 29/300
156/156  3s 18ms/step - loss: 65534.1016 - root_mean_squared_error: 255.9942 - val_loss: 68261.2109 - val_root_mean_squared_error: 261.2664
Epoch 30/300
156/156  3s 16ms/step - loss: 65032.5117 - root_mean_squared_error: 255.0125 - val_loss: 70389.0781 - val_root_mean_squared_error: 265.3073
Epoch 31/300
156/156  2s 13ms/step - loss: 66305.8359 - root_mean_squared_error: 257.4970 - val_loss: 68414.4375 - val_root_mean_squared_error: 261.5594
Epoch 32/300
156/156  2s 11ms/step - loss: 65315.9336 - root_mean_squared_error: 255.5676 - val_loss: 69677.1484 - val_root_mean_squared_error: 263.9622
Epoch 33/300
156/156  1s 9ms/step - loss: 64452.8750 - root_mean_squared_error: 253.8735 - val_loss: 66765.5781 - val_root_mean_squared_error: 258.3882
Epoch 34/300
156/156  2s 9ms/step - loss: 63543.0391 - root_mean_squared_error: 252.0752 - val_loss: 68795.9375 - val_root_mean_squared_error: 262.2877
Epoch 35/300
156/156  1s 9ms/step - loss: 63799.7812 - root_mean_squared_error: 252.5840 - val_loss: 69530.3203 - val_root_mean_squared_error: 263.6839
Epoch 36/300
156/156  1s 9ms/step - loss: 61494.5312 - root_mean_squared_error: 247.9786 - val_loss: 65641.2344 - val_root_mean_squared_error: 256.2032
Epoch 37/300
156/156  2s 13ms/step - loss: 63426.7969 - root_mean_squared_error: 251.8445 - val_loss: 66397.7031 - val_root_mean_squared_error: 257.6753
Epoch 38/300
156/156  2s 10ms/step - loss: 61581.1523 - root_mean_squared_error: 248.1532 - val_loss: 66698.6719 - val_root_mean_squared_error: 258.2586
Epoch 39/300
156/156  2s 11ms/step - loss: 61186.6367 - root_mean_squared_error: 247.3570 - val_loss: 65764.2578 - val_root_mean_squared_error: 256.4432
Epoch 40/300
156/156  1s 9ms/step - loss: 59106.7969 - root_mean_squared_error: 243.1165 - val_loss: 67602.2812 - val_root_mean_squared_error: 260.0021
Epoch 41/300
156/156  1s 9ms/step - loss: 60268.2773 - root_mean_squared_error: 245.4936 - val_loss: 70634.9688 - val_root_mean_squared_error: 265.7702
Epoch 42/300
156/156  1s 9ms/step - loss: 58054.0352 - root_mean_squared_error: 240.9416 - val_loss: 65503.0781 - val_root_mean_squared_error: 255.9334
Epoch 43/300
156/156  1s 9ms/step - loss: 58756.7031 - root_mean_squared_error: 242.3954 - val_loss: 66444.5156 - val_root_mean_squared_error: 257.7660
Epoch 44/300
156/156  1s 8ms/step - loss: 59348.6016 - root_mean_squared_error: 243.6133 - val_loss: 66248.4062 - val_root_mean_squared_error: 257.3853
Epoch 45/300
156/156  1s 9ms/step - loss: 60502.4922 - root_mean_squared_error: 245.9701 - val_loss: 65400.8906 - val_root_mean_squared_error: 255.7336
Epoch 46/300
156/156  1s 9ms/step - loss: 59136.3672 - root_mean_squared_error: 243.1772 - val_loss: 64732.6172 - val_root_mean_squared_error: 254.4237
Epoch 47/300
156/156  2s 14ms/step - loss: 59026.1797 - root_mean_squared_error: 242.9506 - val_loss: 65669.5078 - val_root_mean_squared_error: 256.2582
Epoch 48/300
156/156  3s 20ms/step - loss: 56108.7539 - root_mean_squared_error: 236.8703 - val_loss: 66919.5391 - val_root_mean_squared_error: 258.6858
Epoch 49/300
156/156  3s 16ms/step - loss: 57565.7109 - root_mean_squared_error: 239.9260 - val_loss: 65808.3281 - val_root_mean_squared_error: 256.5289
Epoch 50/300
156/156  2s 10ms/step - loss: 57290.9961 - root_mean_squared_error: 239.3529 - val_loss: 66952.1172 - val_root_mean_squared_error: 258.7487
Epoch 51/300
156/156  1s 9ms/step - loss: 57307.5977 - root_mean_squared_error: 239.3875 - val_loss: 64597.4805 - val_root_mean_squared_error: 254.1579
Epoch 52/300
156/156  1s 9ms/step - loss: 53545.5508 - root_mean_squared_error: 231.3964 - val_loss: 64987.3242 - val_root_mean_squared_error: 254.9237
Epoch 53/300
156/156  1s 9ms/step - loss: 55670.1094 - root_mean_squared_error: 235.9425 - val_loss: 63464.2891 - val_root_mean_squared_error: 251.9187
Epoch 54/300
156/156  2s 14ms/step - loss: 55366.5703 - root_mean_squared_error: 235.2984 - val_loss: 65443.6680 - val_root_mean_squared_error: 255.8172
Epoch 55/300
156/156  2s 9ms/step - loss: 55365.5312 - root_mean_squared_error: 235.2962 - val_loss: 63725.2539 - val_root_mean_squared_error: 252.4361
Epoch 56/300
156/156  2s 10ms/step - loss: 53413.7500 - root_mean_squared_error: 231.1115 - val_loss: 62574.4023 - val_root_mean_squared_error: 250.1462

Epoch 57/300
156/156  1s 9ms/step - loss: 55338.2344 - root_mean_squared_error: 235.2382 - val_loss: 64498.3867 - val_root_mean_squared_error: 253.9629
Epoch 58/300
156/156  2s 9ms/step - loss: 53328.6562 - root_mean_squared_error: 230.9273 - val_loss: 63436.6562 - val_root_mean_squared_error: 251.8638
Epoch 59/300
156/156  1s 9ms/step - loss: 54336.9375 - root_mean_squared_error: 233.1001 - val_loss: 63053.0625 - val_root_mean_squared_error: 251.1012
Epoch 60/300
156/156  2s 12ms/step - loss: 55603.6133 - root_mean_squared_error: 235.8015 - val_loss: 65375.2812 - val_root_mean_squared_error: 255.6834
Epoch 61/300
156/156  2s 11ms/step - loss: 53034.1367 - root_mean_squared_error: 230.2887 - val_loss: 64500.7734 - val_root_mean_squared_error: 253.9675
Epoch 62/300
156/156  2s 9ms/step - loss: 52972.6719 - root_mean_squared_error: 230.1552 - val_loss: 65824.6797 - val_root_mean_squared_error: 256.5607
Epoch 63/300
156/156  2s 14ms/step - loss: 53555.4844 - root_mean_squared_error: 231.4178 - val_loss: 62451.0547 - val_root_mean_squared_error: 249.8995
Epoch 64/300
156/156  2s 13ms/step - loss: 52986.5391 - root_mean_squared_error: 230.1852 - val_loss: 66204.6406 - val_root_mean_squared_error: 257.3001
Epoch 65/300
156/156  3s 16ms/step - loss: 53266.5078 - root_mean_squared_error: 230.7926 - val_loss: 64799.9922 - val_root_mean_squared_error: 254.5559
Epoch 66/300
156/156  2s 14ms/step - loss: 53424.3320 - root_mean_squared_error: 231.1342 - val_loss: 65628.1797 - val_root_mean_squared_error: 256.1774
Epoch 67/300
156/156  2s 10ms/step - loss: 52284.1211 - root_mean_squared_error: 228.6544 - val_loss: 65049.7852 - val_root_mean_squared_error: 255.0460
Epoch 68/300
156/156  1s 9ms/step - loss: 52566.1445 - root_mean_squared_error: 229.2702 - val_loss: 66346.3359 - val_root_mean_squared_error: 257.5753
Epoch 69/300
156/156  1s 9ms/step - loss: 53413.2812 - root_mean_squared_error: 231.1103 - val_loss: 64618.0000 - val_root_mean_squared_error: 254.1981
Epoch 70/300
156/156  1s 9ms/step - loss: 52104.8789 - root_mean_squared_error: 228.2620 - val_loss: 63021.6016 - val_root_mean_squared_error: 251.0384
Epoch 71/300
156/156  2s 11ms/step - loss: 52301.6211 - root_mean_squared_error: 228.6926 - val_loss: 63641.8750 - val_root_mean_squared_error: 252.2708
Epoch 72/300
156/156  2s 10ms/step - loss: 51128.7773 - root_mean_squared_error: 226.1138 - val_loss: 61926.8711 - val_root_mean_squared_error: 248.8484
Epoch 73/300
156/156  2s 13ms/step - loss: 51704.9062 - root_mean_squared_error: 227.3842 - val_loss: 63815.8984 - val_root_mean_squared_error: 252.6155
Epoch 74/300
156/156  2s 11ms/step - loss: 52562.0078 - root_mean_squared_error: 229.2612 - val_loss: 63788.6641 - val_root_mean_squared_error: 252.5616
Epoch 75/300
156/156  1s 9ms/step - loss: 50902.5078 - root_mean_squared_error: 225.6129 - val_loss: 62133.0352 - val_root_mean_squared_error: 249.2623
Epoch 76/300
156/156  1s 9ms/step - loss: 50288.4336 - root_mean_squared_error: 224.2478 - val_loss: 64978.4844 - val_root_mean_squared_error: 254.9061
Epoch 77/300
156/156  1s 9ms/step - loss: 50838.5859 - root_mean_squared_error: 225.4712 - val_loss: 63964.1133 - val_root_mean_squared_error: 252.9086
Epoch 78/300
156/156  1s 9ms/step - loss: 49237.2578 - root_mean_squared_error: 221.8917 - val_loss: 63189.5938 - val_root_mean_squared_error: 251.3728
Epoch 79/300
156/156  1s 9ms/step - loss: 50720.3828 - root_mean_squared_error: 225.2088 - val_loss: 67574.5078 - val_root_mean_squared_error: 259.9484
Epoch 80/300
156/156  2s 9ms/step - loss: 49302.1602 - root_mean_squared_error: 222.0379 - val_loss: 63085.5117 - val_root_mean_squared_error: 251.1656
Epoch 81/300
156/156  2s 10ms/step - loss: 49761.6523 - root_mean_squared_error: 223.0701 - val_loss: 61906.9141 - val_root_mean_squared_error: 248.8083
Epoch 82/300
156/156  2s 12ms/step - loss: 49495.0195 - root_mean_squared_error: 222.4717 - val_loss: 61772.0625 - val_root_mean_squared_error: 248.5371
Epoch 83/300
156/156  2s 15ms/step - loss: 51030.3281 - root_mean_squared_error: 225.8958 - val_loss: 62288.8789 - val_root_mean_squared_error: 249.5747
Epoch 84/300
156/156  3s 17ms/step - loss: 49444.3516 - root_mean_squared_error: 222.3578 - val_loss: 63643.9766 - val_root_mean_squared_error: 252.2749

Epoch 85/300
156/156  2s 11ms/step - loss: 48001.5352 - root_mean_squared_error: 219.0894 - val_loss: 62273.7383 - val_root_mean_squared_error: 249.5443
Epoch 86/300
156/156  2s 10ms/step - loss: 49047.1992 - root_mean_squared_error: 221.4629 - val_loss: 63078.2852 - val_root_mean_squared_error: 251.1512
Epoch 87/300
156/156  3s 13ms/step - loss: 48959.1289 - root_mean_squared_error: 221.2639 - val_loss: 66354.2969 - val_root_mean_squared_error: 257.5906
Epoch 88/300
156/156  2s 9ms/step - loss: 49353.6211 - root_mean_squared_error: 222.1536 - val_loss: 62963.3828 - val_root_mean_squared_error: 250.9223
Epoch 89/300
156/156  2s 10ms/step - loss: 49220.5352 - root_mean_squared_error: 221.8539 - val_loss: 64549.8203 - val_root_mean_squared_error: 254.0638
Epoch 90/300
156/156  2s 10ms/step - loss: 48194.2539 - root_mean_squared_error: 219.5287 - val_loss: 65771.8672 - val_root_mean_squared_error: 256.4575
Epoch 91/300
156/156  1s 9ms/step - loss: 48558.3398 - root_mean_squared_error: 220.3564 - val_loss: 63439.8672 - val_root_mean_squared_error: 251.8699
Epoch 92/300
156/156  1s 9ms/step - loss: 49485.5859 - root_mean_squared_error: 222.4503 - val_loss: 66199.9141 - val_root_mean_squared_error: 257.2907
Epoch 93/300
156/156  2s 10ms/step - loss: 47022.2930 - root_mean_squared_error: 216.8430 - val_loss: 62916.7422 - val_root_mean_squared_error: 250.8293
Epoch 94/300
156/156  2s 10ms/step - loss: 47054.4258 - root_mean_squared_error: 216.9171 - val_loss: 64175.3438 - val_root_mean_squared_error: 253.3257
Epoch 95/300
156/156  2s 11ms/step - loss: 48513.4805 - root_mean_squared_error: 220.2545 - val_loss: 64469.4258 - val_root_mean_squared_error: 253.9055
Epoch 96/300
156/156  2s 12ms/step - loss: 48123.7305 - root_mean_squared_error: 219.3680 - val_loss: 63255.9805 - val_root_mean_squared_error: 251.5046
Epoch 97/300
156/156  2s 12ms/step - loss: 47968.3438 - root_mean_squared_error: 219.0135 - val_loss: 62103.8125 - val_root_mean_squared_error: 249.2035
Epoch 98/300
156/156  2s 9ms/step - loss: 47732.5664 - root_mean_squared_error: 218.4746 - val_loss: 61595.2266 - val_root_mean_squared_error: 248.1810
Epoch 99/300
156/156  3s 16ms/step - loss: 47462.3047 - root_mean_squared_error: 217.8551 - val_loss: 60494.5898 - val_root_mean_squared_error: 245.9536
Epoch 100/300
156/156  3s 17ms/step - loss: 46936.1445 - root_mean_squared_error: 216.6442 - val_loss: 61977.0273 - val_root_mean_squared_error: 248.9490
Epoch 101/300
156/156  2s 14ms/step - loss: 46311.1719 - root_mean_squared_error: 215.1970 - val_loss: 62071.9609 - val_root_mean_squared_error: 249.1396
Epoch 102/300
156/156  2s 11ms/step - loss: 48142.3398 - root_mean_squared_error: 219.4104 - val_loss: 62010.8945 - val_root_mean_squared_error: 249.0170
Epoch 103/300
156/156  1s 9ms/step - loss: 47548.3359 - root_mean_squared_error: 218.0525 - val_loss: 61781.2852 - val_root_mean_squared_error: 248.5555
Epoch 104/300
156/156  2s 12ms/step - loss: 46846.2070 - root_mean_squared_error: 216.4365 - val_loss: 62259.2852 - val_root_mean_squared_error: 249.5152
Epoch 105/300
156/156  1s 9ms/step - loss: 46868.6562 - root_mean_squared_error: 216.4883 - val_loss: 60881.5508 - val_root_mean_squared_error: 246.7389
Epoch 106/300
156/156  2s 11ms/step - loss: 47715.1133 - root_mean_squared_error: 218.4346 - val_loss: 61723.6562 - val_root_mean_squared_error: 248.4395
Epoch 107/300
156/156  1s 9ms/step - loss: 46280.0742 - root_mean_squared_error: 215.1246 - val_loss: 60209.0586 - val_root_mean_squared_error: 245.3723
Epoch 108/300
156/156  2s 10ms/step - loss: 47843.2227 - root_mean_squared_error: 218.7275 - val_loss: 60512.6172 - val_root_mean_squared_error: 245.9901
Epoch 109/300
156/156  3s 10ms/step - loss: 46943.0430 - root_mean_squared_error: 216.6600 - val_loss: 61102.9727 - val_root_mean_squared_error: 247.1872
Epoch 110/300
156/156  2s 9ms/step - loss: 45308.8203 - root_mean_squared_error: 212.8552 - val_loss: 60023.2188 - val_root_mean_squared_error: 256.9470
Epoch 111/300
156/156  1s 9ms/step - loss: 46090.6836 - root_mean_squared_error: 214.6840 - val_loss: 63452.0977 - val_root_mean_squared_error: 251.8941
Epoch 112/300
156/156  1s 9ms/step - loss: 44859.0234 - root_mean_squared_error: 211.7960 - val_loss: 62787.3828 - val_root_mean_squared_error: 250.5711

Epoch 113/300
156/156 ————— 1s 9ms/step - loss: 45609.4258 - root_mean_squared_error: 213.5601 - val_loss: 63372.7461 - val_root_mean_squared_error: 251.7365
Epoch 114/300
156/156 ————— 1s 9ms/step - loss: 44314.8789 - root_mean_squared_error: 210.5074 - val_loss: 61390.9219 - val_root_mean_squared_error: 247.7689
Epoch 115/300
156/156 ————— 2s 11ms/step - loss: 45776.2969 - root_mean_squared_error: 213.9505 - val_loss: 64142.6016 - val_root_mean_squared_error: 253.2609
Epoch 116/300
156/156 ————— 3s 17ms/step - loss: 46071.1367 - root_mean_squared_error: 214.6384 - val_loss: 62215.2500 - val_root_mean_squared_error: 249.4268
Epoch 117/300
156/156 ————— 3s 16ms/step - loss: 43729.9062 - root_mean_squared_error: 209.1134 - val_loss: 61364.3359 - val_root_mean_squared_error: 247.7152
Epoch 118/300
156/156 ————— 3s 17ms/step - loss: 44949.4219 - root_mean_squared_error: 212.0092 - val_loss: 63392.8906 - val_root_mean_squared_error: 251.7764
Epoch 119/300
156/156 ————— 1s 9ms/step - loss: 44862.5703 - root_mean_squared_error: 211.8043 - val_loss: 66014.6562 - val_root_mean_squared_error: 256.9302
Epoch 120/300
156/156 ————— 2s 11ms/step - loss: 45580.3672 - root_mean_squared_error: 213.4920 - val_loss: 61592.2969 - val_root_mean_squared_error: 248.1749
Epoch 121/300
156/156 ————— 2s 10ms/step - loss: 43544.0781 - root_mean_squared_error: 208.6685 - val_loss: 60159.9180 - val_root_mean_squared_error: 245.2720
Epoch 122/300
156/156 ————— 2s 10ms/step - loss: 46486.4141 - root_mean_squared_error: 215.6035 - val_loss: 63429.9727 - val_root_mean_squared_error: 251.8500
Epoch 123/300
156/156 ————— 2s 9ms/step - loss: 45390.2656 - root_mean_squared_error: 213.0463 - val_loss: 63433.4766 - val_root_mean_squared_error: 251.8570
Epoch 124/300
156/156 ————— 2s 11ms/step - loss: 43247.9922 - root_mean_squared_error: 207.9578 - val_loss: 60485.9844 - val_root_mean_squared_error: 245.9359
Epoch 125/300
156/156 ————— 1s 9ms/step - loss: 44680.4219 - root_mean_squared_error: 211.3738 - val_loss: 65490.8867 - val_root_mean_squared_error: 255.9089
Epoch 126/300
156/156 ————— 2s 11ms/step - loss: 44412.7773 - root_mean_squared_error: 210.7398 - val_loss: 60454.0898 - val_root_mean_squared_error: 245.8710
Epoch 127/300
156/156 ————— 1s 9ms/step - loss: 44281.4570 - root_mean_squared_error: 210.4279 - val_loss: 59575.7305 - val_root_mean_squared_error: 244.0782
Epoch 128/300
156/156 ————— 3s 13ms/step - loss: 42503.6875 - root_mean_squared_error: 206.1604 - val_loss: 57839.5312 - val_root_mean_squared_error: 240.4953
Epoch 129/300
156/156 ————— 1s 9ms/step - loss: 43893.6992 - root_mean_squared_error: 209.5045 - val_loss: 61480.7461 - val_root_mean_squared_error: 247.9500
Epoch 130/300
156/156 ————— 2s 10ms/step - loss: 44211.5156 - root_mean_squared_error: 210.2617 - val_loss: 61974.4414 - val_root_mean_squared_error: 248.9435
Epoch 131/300
156/156 ————— 2s 10ms/step - loss: 44932.4727 - root_mean_squared_error: 211.9691 - val_loss: 60802.4492 - val_root_mean_squared_error: 246.5783
Epoch 132/300
156/156 ————— 2s 10ms/step - loss: 42393.9766 - root_mean_squared_error: 205.8941 - val_loss: 61844.8320 - val_root_mean_squared_error: 248.6830
Epoch 133/300
156/156 ————— 3s 16ms/step - loss: 42735.1680 - root_mean_squared_error: 206.7211 - val_loss: 59757.3438 - val_root_mean_squared_error: 244.4500
Epoch 134/300
156/156 ————— 3s 16ms/step - loss: 43043.6680 - root_mean_squared_error: 207.4659 - val_loss: 59179.8672 - val_root_mean_squared_error: 243.2659
Epoch 135/300
156/156 ————— 2s 12ms/step - loss: 43573.0625 - root_mean_squared_error: 208.7378 - val_loss: 60803.0664 - val_root_mean_squared_error: 246.5796
Epoch 136/300
156/156 ————— 2s 11ms/step - loss: 43470.5039 - root_mean_squared_error: 208.4920 - val_loss: 59261.5703 - val_root_mean_squared_error: 243.4337
Epoch 137/300
156/156 ————— 1s 9ms/step - loss: 43145.5586 - root_mean_squared_error: 207.7112 - val_loss: 61196.3164 - val_root_mean_squared_error: 247.3757
Epoch 138/300
156/156 ————— 2s 12ms/step - loss: 43551.3828 - root_mean_squared_error: 208.6859 - val_loss: 63655.6602 - val_root_mean_squared_error: 252.2975
Epoch 139/300
156/156 ————— 1s 9ms/step - loss: 44029.4062 - root_mean_squared_error: 209.8281 - val_loss: 72227.1172 - val_root_mean_squared_error: 268.7481
Epoch 140/300
156/156 ————— 2s 10ms/step - loss: 43480.0234 - root_mean_squared_error: 208.5148 - val_loss: 62426.1250 - val_root_mean_squared_error: 249.8490

```

Epoch 141/300
156/156 ————— 1s 9ms/step - loss: 43374.9492 - root_mean_squared_error: 208.2627 - val_loss: 60863.2383 - val_root_mean
_squared_error: 246.7015
Epoch 142/300
156/156 ————— 1s 9ms/step - loss: 43210.3164 - root_mean_squared_error: 207.8670 - val_loss: 60069.6953 - val_root_mean
_squared_error: 245.0879
Epoch 143/300
156/156 ————— 1s 9ms/step - loss: 41003.5156 - root_mean_squared_error: 202.4893 - val_loss: 60685.3750 - val_root_mean
_squared_error: 246.3407
Epoch 144/300
156/156 ————— 2s 9ms/step - loss: 41972.0820 - root_mean_squared_error: 204.8670 - val_loss: 60505.0273 - val_root_mean
_squared_error: 245.9744
Epoch 145/300
156/156 ————— 2s 9ms/step - loss: 43739.0977 - root_mean_squared_error: 209.1351 - val_loss: 61009.5000 - val_root_mean
_squared_error: 246.9978
Epoch 146/300
156/156 ————— 2s 12ms/step - loss: 42212.1562 - root_mean_squared_error: 205.4520 - val_loss: 64758.4023 - val_root_mean
_squared_error: 254.4736
Epoch 147/300
156/156 ————— 2s 11ms/step - loss: 42514.6914 - root_mean_squared_error: 206.1870 - val_loss: 63230.6406 - val_root_mean
_squared_error: 251.4538
Epoch 148/300
156/156 ————— 2s 9ms/step - loss: 43891.9492 - root_mean_squared_error: 209.5001 - val_loss: 64100.9141 - val_root_mean
_squared_error: 253.1783

```

```

Out[30]: {'model': 'Neural Network',
          'best_params': {'epochs': 300, 'batch_size': 64},
          'cv_rmse': None,
          'train_rmse': 178.1974728997621,
          'test_rmse': 221.42547859980695,
          'train_r2': 0.9423158078085699,
          'test_r2': 0.9061021519495637,
          'test_mae': 142.04742252058674,
          'overfit_gap_r2': 0.036213655859006266}

```

The Neural Network that we modeled performs better than most of the models, but still it doesn't achieve the results of XGBoost and RandomForest. During training we notice that it converges around 148 epochs, so we cannot improve it further unless we model a better NN.

Part 2: Trying models with partial data set

During EDA and other analysis we noticed that some of the features had unexpected effect or not a very clear effect, so I would say, it would be best to also try training some ML model without these features and see how they perform. We might find out that the model might be better of without these noisy features and it also makes it better for the users not having to add every single unnecessary detail about the apartment.

First we try it without the features that had an unexpected opposite effect.

```

In [31]: features_reduced = [
          c for c in X_train.columns
          if c not in ["has_bathtub", "has_terrace"]
        ]

X_train_2 = X_train[features_reduced]
X_test_2 = X_test[features_reduced]

In [32]: features_2 = X_train_2.columns.tolist()

metrics_rf_2, bestmodel_rf_2 = evaluate_model(
    "Random Forest - no bathtub/terrace",
    RandomForestRegressor(random_state=42),
    {
        "model__n_estimators": [100, 200, 300, 400],
        "model__max_depth": [None, 10, 20, 30, 40],
        "model__min_samples_split": [2, 5, 10, 20],
        "model__min_samples_leaf": [1, 2, 4, 8],
        "model__max_features": ["sqrt", "log2"],
    },
    X_train_2,
    y_train,
    X_test_2,
    y_test,
    feature_cols=features_2
)

metrics_rf_2

```

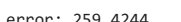
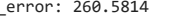
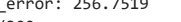
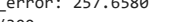
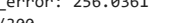
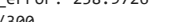
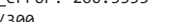
```
Out[32]: {'model': 'Random Forest - no bathtub/terrace',
          'best_params': {'model__max_depth': 40,
                           'model__max_features': 'sqrt',
                           'model__min_samples_leaf': 1,
                           'model__min_samples_split': 2,
                           'model__n_estimators': 400},
          'cv_rmse': 234.2207994033461,
          'train_rmse': 95.45431000119082,
          'test_rmse': 206.311561869115,
          'train_r2': 0.9834482156697438,
          'test_r2': 0.9184831125194224,
          'test_mae': 123.62733364823701,
          'overfit_gap_r2': 0.06496510315032145}
```

Random forest seems to be performing slightly better so it seems that it would be better to not train our models without these features.

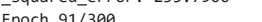
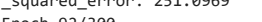
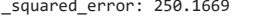
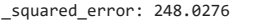
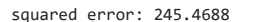
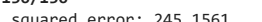
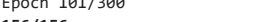
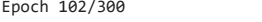
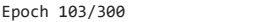
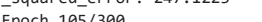
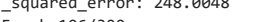
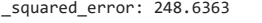
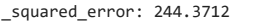
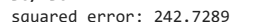
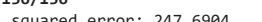
```
In [33]: metrics_nn_2, bestmodel_nn_2, nn_pipeline_2, history_nn_2 = evaluate_nn(
          X_train_2,
          y_train,
          X_test_2,
          y_test,
          epochs=300,
          batch_size=64,
          feature_cols=features_2
        )

metrics_nn_2
```

Epoch 1/300
156/156 ————— 3s 5ms/step - loss: 1827570.0000 - root_mean_squared_error: 1351.8762 - val_loss: 1484833.6250 - val_root_mean_squared_error: 1218.5374
Epoch 2/300
156/156 ————— 1s 3ms/step - loss: 480540.7500 - root_mean_squared_error: 693.2100 - val_loss: 294701.6562 - val_root_mean_squared_error: 542.8636
Epoch 3/300
156/156 ————— 1s 3ms/step - loss: 227515.6875 - root_mean_squared_error: 476.9853 - val_loss: 186626.1875 - val_root_mean_squared_error: 432.0017
Epoch 4/300
156/156 ————— 1s 3ms/step - loss: 189834.6094 - root_mean_squared_error: 435.6993 - val_loss: 140226.3281 - val_root_mean_squared_error: 374.4671
Epoch 5/300
156/156 ————— 0s 3ms/step - loss: 156056.7188 - root_mean_squared_error: 395.0392 - val_loss: 112875.5078 - val_root_mean_squared_error: 335.9683
Epoch 6/300
156/156 ————— 0s 3ms/step - loss: 132211.4531 - root_mean_squared_error: 363.6079 - val_loss: 102048.2656 - val_root_mean_squared_error: 319.4487
Epoch 7/300
156/156 ————— 1s 5ms/step - loss: 119072.7734 - root_mean_squared_error: 345.0681 - val_loss: 95470.6406 - val_root_mean_squared_error: 308.9819
Epoch 8/300
156/156 ————— 1s 5ms/step - loss: 111608.5312 - root_mean_squared_error: 334.0773 - val_loss: 90677.2656 - val_root_mean_squared_error: 301.1253
Epoch 9/300
156/156 ————— 1s 3ms/step - loss: 104773.0078 - root_mean_squared_error: 323.6853 - val_loss: 87355.9922 - val_root_mean_squared_error: 295.5591
Epoch 10/300
156/156 ————— 1s 6ms/step - loss: 100987.8047 - root_mean_squared_error: 317.7845 - val_loss: 83464.9531 - val_root_mean_squared_error: 288.9016
Epoch 11/300
156/156 ————— 1s 4ms/step - loss: 95954.4688 - root_mean_squared_error: 309.7638 - val_loss: 81738.4219 - val_root_mean_squared_error: 285.8978
Epoch 12/300
156/156 ————— 0s 3ms/step - loss: 91720.8594 - root_mean_squared_error: 302.8531 - val_loss: 84685.5781 - val_root_mean_squared_error: 291.0064
Epoch 13/300
156/156 ————— 1s 3ms/step - loss: 90068.0703 - root_mean_squared_error: 300.1121 - val_loss: 82519.0156 - val_root_mean_squared_error: 287.2597
Epoch 14/300
156/156 ————— 1s 4ms/step - loss: 89407.5469 - root_mean_squared_error: 299.0095 - val_loss: 80287.8672 - val_root_mean_squared_error: 283.3496
Epoch 15/300
156/156 ————— 1s 3ms/step - loss: 83032.7344 - root_mean_squared_error: 288.1525 - val_loss: 79211.8750 - val_root_mean_squared_error: 281.4444
Epoch 16/300
156/156 ————— 0s 3ms/step - loss: 84021.3516 - root_mean_squared_error: 289.8629 - val_loss: 76865.0547 - val_root_mean_squared_error: 277.2439
Epoch 17/300
156/156 ————— 0s 3ms/step - loss: 82088.6250 - root_mean_squared_error: 286.5096 - val_loss: 74341.1406 - val_root_mean_squared_error: 272.6541
Epoch 18/300
156/156 ————— 0s 3ms/step - loss: 79180.0156 - root_mean_squared_error: 281.3878 - val_loss: 75430.4219 - val_root_mean_squared_error: 274.6443
Epoch 19/300
156/156 ————— 0s 3ms/step - loss: 77165.3047 - root_mean_squared_error: 277.7848 - val_loss: 74043.0547 - val_root_mean_squared_error: 272.1068
Epoch 20/300
156/156 ————— 0s 3ms/step - loss: 75157.2188 - root_mean_squared_error: 274.1465 - val_loss: 75278.1562 - val_root_mean_squared_error: 274.3669
Epoch 21/300
156/156 ————— 0s 3ms/step - loss: 74908.2266 - root_mean_squared_error: 273.6920 - val_loss: 75346.6328 - val_root_mean_squared_error: 274.4917
Epoch 22/300
156/156 ————— 0s 3ms/step - loss: 72906.2109 - root_mean_squared_error: 270.0098 - val_loss: 73536.4297 - val_root_mean_squared_error: 271.1743
Epoch 23/300
156/156 ————— 0s 3ms/step - loss: 71016.1328 - root_mean_squared_error: 266.4868 - val_loss: 75775.3047 - val_root_mean_squared_error: 275.2714
Epoch 24/300
156/156 ————— 0s 3ms/step - loss: 69718.3203 - root_mean_squared_error: 264.0405 - val_loss: 72771.8594 - val_root_mean_squared_error: 269.7608
Epoch 25/300
156/156 ————— 1s 3ms/step - loss: 70201.9688 - root_mean_squared_error: 264.9547 - val_loss: 70692.5703 - val_root_mean_squared_error: 265.8789
Epoch 26/300
156/156 ————— 1s 4ms/step - loss: 71392.7500 - root_mean_squared_error: 267.1924 - val_loss: 73348.2109 - val_root_mean_squared_error: 270.8270
Epoch 27/300
156/156 ————— 1s 3ms/step - loss: 68864.4844 - root_mean_squared_error: 262.4186 - val_loss: 71087.1328 - val_root_mean_squared_error: 266.6199
Epoch 28/300
156/156 ————— 1s 4ms/step - loss: 68251.4844 - root_mean_squared_error: 261.2480 - val_loss: 73397.0859 - val_root_mean_squared_error: 270.9172

Epoch 29/300
156/156  1s 3ms/step - loss: 67210.0625 - root_mean_squared_error: 259.2472 - val_loss: 68927.3281 - val_root_mean_squared_error: 262.5383
Epoch 30/300
156/156  1s 4ms/step - loss: 66141.1172 - root_mean_squared_error: 257.1772 - val_loss: 72377.9609 - val_root_mean_squared_error: 269.0297
Epoch 31/300
156/156  0s 3ms/step - loss: 65788.1797 - root_mean_squared_error: 256.4902 - val_loss: 71313.3125 - val_root_mean_squared_error: 267.0437
Epoch 32/300
156/156  0s 3ms/step - loss: 64390.3047 - root_mean_squared_error: 253.7505 - val_loss: 68015.6641 - val_root_mean_squared_error: 260.7962
Epoch 33/300
156/156  0s 3ms/step - loss: 62241.5859 - root_mean_squared_error: 249.4806 - val_loss: 67687.6328 - val_root_mean_squared_error: 260.1665
Epoch 34/300
156/156  0s 3ms/step - loss: 62898.9961 - root_mean_squared_error: 250.7947 - val_loss: 71119.8281 - val_root_mean_squared_error: 266.6811
Epoch 35/300
156/156  0s 3ms/step - loss: 65520.5078 - root_mean_squared_error: 255.9678 - val_loss: 68874.0234 - val_root_mean_squared_error: 262.4367
Epoch 36/300
156/156  1s 3ms/step - loss: 61931.3047 - root_mean_squared_error: 248.8580 - val_loss: 69227.5781 - val_root_mean_squared_error: 263.1094
Epoch 37/300
156/156  1s 3ms/step - loss: 62444.9062 - root_mean_squared_error: 249.8877 - val_loss: 67673.9297 - val_root_mean_squared_error: 260.1402
Epoch 38/300
156/156  1s 3ms/step - loss: 64009.1289 - root_mean_squared_error: 252.9982 - val_loss: 69984.0391 - val_root_mean_squared_error: 264.5430
Epoch 39/300
156/156  1s 3ms/step - loss: 59681.4375 - root_mean_squared_error: 244.2957 - val_loss: 67302.0469 - val_root_mean_squared_error: 259.4244
Epoch 40/300
156/156  1s 3ms/step - loss: 61187.3828 - root_mean_squared_error: 247.3588 - val_loss: 68148.7734 - val_root_mean_squared_error: 261.0512
Epoch 41/300
156/156  1s 3ms/step - loss: 63565.2461 - root_mean_squared_error: 252.1194 - val_loss: 68691.8125 - val_root_mean_squared_error: 262.0893
Epoch 42/300
156/156  1s 3ms/step - loss: 59501.8047 - root_mean_squared_error: 243.9278 - val_loss: 70379.2969 - val_root_mean_squared_error: 265.2890
Epoch 43/300
156/156  1s 3ms/step - loss: 59560.2383 - root_mean_squared_error: 244.0475 - val_loss: 70020.6328 - val_root_mean_squared_error: 264.6122
Epoch 44/300
156/156  1s 3ms/step - loss: 59136.4805 - root_mean_squared_error: 243.1777 - val_loss: 67997.6172 - val_root_mean_squared_error: 260.7615
Epoch 45/300
156/156  1s 3ms/step - loss: 58257.0508 - root_mean_squared_error: 241.3628 - val_loss: 67107.5000 - val_root_mean_squared_error: 259.0491
Epoch 46/300
156/156  0s 3ms/step - loss: 58098.8594 - root_mean_squared_error: 241.0349 - val_loss: 70752.2188 - val_root_mean_squared_error: 265.9909
Epoch 47/300
156/156  1s 4ms/step - loss: 59489.1680 - root_mean_squared_error: 243.9018 - val_loss: 67903.7578 - val_root_mean_squared_error: 260.5814
Epoch 48/300
156/156  1s 4ms/step - loss: 57439.8828 - root_mean_squared_error: 239.6640 - val_loss: 65922.5938 - val_root_mean_squared_error: 256.7519
Epoch 49/300
156/156  0s 3ms/step - loss: 55911.8203 - root_mean_squared_error: 236.4545 - val_loss: 66388.7188 - val_root_mean_squared_error: 257.6580
Epoch 50/300
156/156  0s 3ms/step - loss: 57301.0430 - root_mean_squared_error: 239.3741 - val_loss: 65555.5703 - val_root_mean_squared_error: 256.0361
Epoch 51/300
156/156  0s 3ms/step - loss: 56805.9844 - root_mean_squared_error: 238.3378 - val_loss: 67067.8906 - val_root_mean_squared_error: 258.9726
Epoch 52/300
156/156  1s 5ms/step - loss: 54350.8633 - root_mean_squared_error: 233.1304 - val_loss: 67785.9922 - val_root_mean_squared_error: 260.3553
Epoch 53/300
156/156  0s 3ms/step - loss: 56189.4023 - root_mean_squared_error: 237.0407 - val_loss: 66209.9609 - val_root_mean_squared_error: 257.3109
Epoch 54/300
156/156  0s 3ms/step - loss: 56566.9492 - root_mean_squared_error: 237.8358 - val_loss: 67271.9453 - val_root_mean_squared_error: 259.3662
Epoch 55/300
156/156  0s 3ms/step - loss: 55303.9023 - root_mean_squared_error: 235.1655 - val_loss: 66283.0469 - val_root_mean_squared_error: 257.4528
Epoch 56/300
156/156  0s 3ms/step - loss: 54034.8555 - root_mean_squared_error: 232.4516 - val_loss: 65770.4922 - val_root_mean_squared_error: 256.4554

Epoch 57/300
156/156 ————— 0s 3ms/step - loss: 53816.4688 - root_mean_squared_error: 231.9814 - val_loss: 66111.4297 - val_root_mean_squared_error: 257.1193
Epoch 58/300
156/156 ————— 1s 4ms/step - loss: 54581.2461 - root_mean_squared_error: 233.6240 - val_loss: 65932.5000 - val_root_mean_squared_error: 256.7711
Epoch 59/300
156/156 ————— 0s 3ms/step - loss: 53268.3672 - root_mean_squared_error: 230.7970 - val_loss: 65886.6484 - val_root_mean_squared_error: 256.6818
Epoch 60/300
156/156 ————— 0s 3ms/step - loss: 56143.2930 - root_mean_squared_error: 236.9434 - val_loss: 65419.3750 - val_root_mean_squared_error: 255.7699
Epoch 61/300
156/156 ————— 1s 5ms/step - loss: 53066.8477 - root_mean_squared_error: 230.3600 - val_loss: 65061.6445 - val_root_mean_squared_error: 255.0696
Epoch 62/300
156/156 ————— 2s 8ms/step - loss: 52638.5195 - root_mean_squared_error: 229.4284 - val_loss: 63478.7539 - val_root_mean_squared_error: 251.9477
Epoch 63/300
156/156 ————— 2s 11ms/step - loss: 55522.3672 - root_mean_squared_error: 235.6295 - val_loss: 63730.1172 - val_root_mean_squared_error: 252.4460
Epoch 64/300
156/156 ————— 2s 10ms/step - loss: 52771.7188 - root_mean_squared_error: 229.7184 - val_loss: 66544.3750 - val_root_mean_squared_error: 257.9597
Epoch 65/300
156/156 ————— 1s 6ms/step - loss: 52960.7891 - root_mean_squared_error: 230.1296 - val_loss: 64910.3047 - val_root_mean_squared_error: 254.7728
Epoch 66/300
156/156 ————— 1s 4ms/step - loss: 53301.6172 - root_mean_squared_error: 230.8689 - val_loss: 65798.9062 - val_root_mean_squared_error: 256.5107
Epoch 67/300
156/156 ————— 1s 3ms/step - loss: 50630.1875 - root_mean_squared_error: 225.0090 - val_loss: 65937.5469 - val_root_mean_squared_error: 256.7808
Epoch 68/300
156/156 ————— 0s 3ms/step - loss: 52859.1484 - root_mean_squared_error: 229.9086 - val_loss: 65747.3984 - val_root_mean_squared_error: 256.4102
Epoch 69/300
156/156 ————— 0s 3ms/step - loss: 52178.3789 - root_mean_squared_error: 228.4234 - val_loss: 65162.8047 - val_root_mean_squared_error: 255.2678
Epoch 70/300
156/156 ————— 0s 3ms/step - loss: 50495.3203 - root_mean_squared_error: 224.7091 - val_loss: 66680.6172 - val_root_mean_squared_error: 258.2236
Epoch 71/300
156/156 ————— 0s 3ms/step - loss: 53583.5898 - root_mean_squared_error: 231.4788 - val_loss: 63515.1953 - val_root_mean_squared_error: 252.0199
Epoch 72/300
156/156 ————— 0s 3ms/step - loss: 51620.1602 - root_mean_squared_error: 227.1981 - val_loss: 61737.8750 - val_root_mean_squared_error: 248.4687
Epoch 73/300
156/156 ————— 1s 3ms/step - loss: 51000.2969 - root_mean_squared_error: 225.8298 - val_loss: 61039.4336 - val_root_mean_squared_error: 247.0592
Epoch 74/300
156/156 ————— 1s 3ms/step - loss: 50535.3828 - root_mean_squared_error: 224.7981 - val_loss: 63800.3203 - val_root_mean_squared_error: 252.5849
Epoch 75/300
156/156 ————— 1s 5ms/step - loss: 48138.6484 - root_mean_squared_error: 219.4025 - val_loss: 62144.2227 - val_root_mean_squared_error: 249.2850
Epoch 76/300
156/156 ————— 1s 4ms/step - loss: 50536.5117 - root_mean_squared_error: 224.8006 - val_loss: 61882.8672 - val_root_mean_squared_error: 248.7603
Epoch 77/300
156/156 ————— 1s 3ms/step - loss: 50004.8359 - root_mean_squared_error: 223.6149 - val_loss: 62830.3242 - val_root_mean_squared_error: 250.6574
Epoch 78/300
156/156 ————— 1s 7ms/step - loss: 48860.8359 - root_mean_squared_error: 221.0421 - val_loss: 63280.4219 - val_root_mean_squared_error: 251.5536
Epoch 79/300
156/156 ————— 1s 5ms/step - loss: 50042.1953 - root_mean_squared_error: 223.6984 - val_loss: 65096.0977 - val_root_mean_squared_error: 255.1370
Epoch 80/300
156/156 ————— 1s 4ms/step - loss: 50021.7148 - root_mean_squared_error: 223.6527 - val_loss: 62901.9336 - val_root_mean_squared_error: 250.8002
Epoch 81/300
156/156 ————— 1s 3ms/step - loss: 50168.0078 - root_mean_squared_error: 223.9794 - val_loss: 60598.5391 - val_root_mean_squared_error: 246.1653
Epoch 82/300
156/156 ————— 1s 3ms/step - loss: 50173.1758 - root_mean_squared_error: 223.9910 - val_loss: 61589.6016 - val_root_mean_squared_error: 248.1701
Epoch 83/300
156/156 ————— 1s 4ms/step - loss: 50377.0312 - root_mean_squared_error: 224.4456 - val_loss: 63925.4609 - val_root_mean_squared_error: 252.8324
Epoch 84/300
156/156 ————— 1s 3ms/step - loss: 49275.5352 - root_mean_squared_error: 221.9782 - val_loss: 61592.2969 - val_root_mean_squared_error: 248.1755

Epoch 85/300
156/156  1s 6ms/step - loss: 48687.4336 - root_mean_squared_error: 220.6495 - val_loss: 62474.0195 - val_root_mean_squared_error: 249.9456
Epoch 86/300
156/156  1s 3ms/step - loss: 49104.1641 - root_mean_squared_error: 221.5918 - val_loss: 66219.7109 - val_root_mean_squared_error: 257.3295
Epoch 87/300
156/156  1s 3ms/step - loss: 50010.5938 - root_mean_squared_error: 223.6276 - val_loss: 62029.8711 - val_root_mean_squared_error: 249.0555
Epoch 88/300
156/156  1s 3ms/step - loss: 49100.1289 - root_mean_squared_error: 221.5827 - val_loss: 64115.0156 - val_root_mean_squared_error: 253.2070
Epoch 89/300
156/156  1s 4ms/step - loss: 47838.5859 - root_mean_squared_error: 218.7175 - val_loss: 62344.5312 - val_root_mean_squared_error: 249.6864
Epoch 90/300
156/156  1s 4ms/step - loss: 47520.2070 - root_mean_squared_error: 217.9884 - val_loss: 65429.8008 - val_root_mean_squared_error: 255.7900
Epoch 91/300
156/156  1s 5ms/step - loss: 48358.3945 - root_mean_squared_error: 219.9026 - val_loss: 63050.9062 - val_root_mean_squared_error: 251.0969
Epoch 92/300
156/156  1s 6ms/step - loss: 46948.4023 - root_mean_squared_error: 216.6729 - val_loss: 62584.7578 - val_root_mean_squared_error: 250.1669
Epoch 93/300
156/156  1s 7ms/step - loss: 47590.3516 - root_mean_squared_error: 218.1492 - val_loss: 61518.9336 - val_root_mean_squared_error: 248.0276
Epoch 94/300
156/156  2s 8ms/step - loss: 47357.4102 - root_mean_squared_error: 217.6147 - val_loss: 60256.1914 - val_root_mean_squared_error: 245.4688
Epoch 95/300
156/156  1s 9ms/step - loss: 47826.2461 - root_mean_squared_error: 218.6891 - val_loss: 60102.7617 - val_root_mean_squared_error: 245.1561
Epoch 96/300
156/156  1s 7ms/step - loss: 47172.7930 - root_mean_squared_error: 217.1900 - val_loss: 60250.5547 - val_root_mean_squared_error: 245.4573
Epoch 97/300
156/156  1s 5ms/step - loss: 47550.0664 - root_mean_squared_error: 218.0569 - val_loss: 60075.4883 - val_root_mean_squared_error: 245.1004
Epoch 98/300
156/156  1s 6ms/step - loss: 48210.4531 - root_mean_squared_error: 219.5658 - val_loss: 59286.1914 - val_root_mean_squared_error: 243.4849
Epoch 99/300
156/156  1s 6ms/step - loss: 47803.2305 - root_mean_squared_error: 218.6365 - val_loss: 62602.8359 - val_root_mean_squared_error: 250.2030
Epoch 100/300
156/156  1s 3ms/step - loss: 47097.6211 - root_mean_squared_error: 217.0169 - val_loss: 59987.6445 - val_root_mean_squared_error: 244.9211
Epoch 101/300
156/156  0s 3ms/step - loss: 46692.2031 - root_mean_squared_error: 216.0808 - val_loss: 62639.8125 - val_root_mean_squared_error: 250.2769
Epoch 102/300
156/156  0s 3ms/step - loss: 47334.1016 - root_mean_squared_error: 217.5610 - val_loss: 61555.4883 - val_root_mean_squared_error: 248.1012
Epoch 103/300
156/156  1s 4ms/step - loss: 46149.6055 - root_mean_squared_error: 214.8216 - val_loss: 60015.5234 - val_root_mean_squared_error: 244.9780
Epoch 104/300
156/156  1s 4ms/step - loss: 45788.3281 - root_mean_squared_error: 213.9790 - val_loss: 61070.7500 - val_root_mean_squared_error: 247.1223
Epoch 105/300
156/156  1s 5ms/step - loss: 46459.8672 - root_mean_squared_error: 215.5425 - val_loss: 61507.6680 - val_root_mean_squared_error: 248.0048
Epoch 106/300
156/156  1s 5ms/step - loss: 45914.2734 - root_mean_squared_error: 214.2731 - val_loss: 61821.3125 - val_root_mean_squared_error: 248.6363
Epoch 107/300
156/156  1s 4ms/step - loss: 46002.1328 - root_mean_squared_error: 214.4780 - val_loss: 59718.6133 - val_root_mean_squared_error: 244.3712
Epoch 108/300
156/156  1s 4ms/step - loss: 46757.5898 - root_mean_squared_error: 216.2320 - val_loss: 58918.6094 - val_root_mean_squared_error: 242.7289
Epoch 109/300
156/156  1s 3ms/step - loss: 45744.1211 - root_mean_squared_error: 213.8756 - val_loss: 61351.8477 - val_root_mean_squared_error: 247.6904
Epoch 110/300
156/156  0s 3ms/step - loss: 46005.3086 - root_mean_squared_error: 214.4854 - val_loss: 57438.9609 - val_root_mean_squared_error: 239.6615
Epoch 111/300
156/156  0s 3ms/step - loss: 45517.3008 - root_mean_squared_error: 213.3447 - val_loss: 60576.8516 - val_root_mean_squared_error: 246.1210
Epoch 112/300
156/156  0s 3ms/step - loss: 45868.1992 - root_mean_squared_error: 214.1655 - val_loss: 62025.5352 - val_root_mean_squared_error: 249.0466


```

Epoch 113/300
156/156 ————— 0s 3ms/step - loss: 46268.4922 - root_mean_squared_error: 215.0980 - val_loss: 62260.5039 - val_root_mean
_squared_error: 249.5179
Epoch 114/300
156/156 ————— 0s 3ms/step - loss: 45345.1016 - root_mean_squared_error: 212.9408 - val_loss: 62499.4531 - val_root_mean
_squared_error: 249.9962
Epoch 115/300
156/156 ————— 0s 3ms/step - loss: 44439.1289 - root_mean_squared_error: 210.8027 - val_loss: 59670.0430 - val_root_mean
_squared_error: 244.2718
Epoch 116/300
156/156 ————— 0s 3ms/step - loss: 44753.8555 - root_mean_squared_error: 211.5479 - val_loss: 60119.4023 - val_root_mean
_squared_error: 245.1898
Epoch 117/300
156/156 ————— 1s 4ms/step - loss: 45682.7891 - root_mean_squared_error: 213.7321 - val_loss: 60494.9492 - val_root_mean
_squared_error: 245.9545
Epoch 118/300
156/156 ————— 1s 3ms/step - loss: 44362.6055 - root_mean_squared_error: 210.6211 - val_loss: 59244.8164 - val_root_mean
_squared_error: 243.3998
Epoch 119/300
156/156 ————— 1s 3ms/step - loss: 45732.9062 - root_mean_squared_error: 213.8494 - val_loss: 61267.8359 - val_root_mean
_squared_error: 247.5207
Epoch 120/300
156/156 ————— 1s 4ms/step - loss: 46473.6953 - root_mean_squared_error: 215.5744 - val_loss: 59936.1875 - val_root_mean
_squared_error: 244.8159
Epoch 121/300
156/156 ————— 1s 3ms/step - loss: 44588.7734 - root_mean_squared_error: 211.1573 - val_loss: 60043.9141 - val_root_mean
_squared_error: 245.0358
Epoch 122/300
156/156 ————— 1s 4ms/step - loss: 44806.9688 - root_mean_squared_error: 211.6733 - val_loss: 62084.6562 - val_root_mean
_squared_error: 249.1652
Epoch 123/300
156/156 ————— 1s 3ms/step - loss: 44474.0039 - root_mean_squared_error: 210.8854 - val_loss: 60212.9141 - val_root_mean
_squared_error: 245.3804
Epoch 124/300
156/156 ————— 1s 4ms/step - loss: 42580.9648 - root_mean_squared_error: 206.3482 - val_loss: 62011.1758 - val_root_mean
_squared_error: 249.0177
Epoch 125/300
156/156 ————— 1s 3ms/step - loss: 43634.5469 - root_mean_squared_error: 208.8856 - val_loss: 60951.0312 - val_root_mean
_squared_error: 246.8799
Epoch 126/300
156/156 ————— 1s 4ms/step - loss: 44805.6367 - root_mean_squared_error: 211.6702 - val_loss: 58463.8672 - val_root_mean
_squared_error: 241.7902
Epoch 127/300
156/156 ————— 1s 5ms/step - loss: 44438.5195 - root_mean_squared_error: 210.8012 - val_loss: 61012.6055 - val_root_mean
_squared_error: 247.0045
Epoch 128/300
156/156 ————— 1s 4ms/step - loss: 43956.9453 - root_mean_squared_error: 209.6558 - val_loss: 59931.8945 - val_root_mean
_squared_error: 244.8071
Epoch 129/300
156/156 ————— 1s 4ms/step - loss: 44854.4336 - root_mean_squared_error: 211.7854 - val_loss: 59526.8906 - val_root_mean
_squared_error: 243.9785
Epoch 130/300
156/156 ————— 1s 4ms/step - loss: 44778.5898 - root_mean_squared_error: 211.6062 - val_loss: 66757.0781 - val_root_mean
_squared_error: 258.3712

```

```

Out[33]: {'model': 'Neural Network',
          'best_params': {'epochs': 300, 'batch_size': 64},
          'cv_rmse': None,
          'train_rmse': 180.64100974059124,
          'test_rmse': 222.5471662644276,
          'train_r2': 0.9407229697047931,
          'test_r2': 0.905148414993896,
          'test_mae': 144.113650951569,
          'overfit_gap_r2': 0.0355745547108971}

```

We cannot reach the same conclusion about the neural network, which performs slightly worse without these 2.

Now we try it by also removing the other noisy features.

```

In [34]: features_reduced = [
          c for c in X_train.columns
          if c not in ["has_bathtub", "has_terrace", "has_balkon", "has_garten", "parking", "has_basement", "is_barrier-free"]
        ]

X_train_3 = X_train[features_reduced]
X_test_3 = X_test[features_reduced]

```

```

In [35]: features_3 = X_train_3.columns.tolist()

metrics_rf_3, bestmodel_rf_3 = evaluate_model(
    "Random Forest - without 7 least important features.",
    RandomForestRegressor(random_state=42),
    {
        "model__n_estimators": [100, 200, 300, 400],
        "model__max_depth": [None, 10, 20, 30, 40],
    }
)

```

```

        "model__min_samples_split": [2, 5, 10, 20],
        "model__min_samples_leaf": [1, 2, 4, 8],
        "model__max_features": ["sqrt", "log2"],
    },
    X_train_3,
    y_train,
    X_test_3,
    y_test,
    feature_cols=features_3
)

metrics_rf_3

```

```

Out[35]: {'model': 'Random Forest - without 7 least important features.',
'best_params': {'model__max_depth': 40,
'model__max_features': 'sqrt',
'model__min_samples_leaf': 1,
'model__min_samples_split': 2,
'model__n_estimators': 400},
'cv_rmse': 233.66988212572176,
'train_rmse': 95.08428516250468,
'test_rmse': 204.44780700829875,
'train_r2': 0.9835762916151471,
'test_r2': 0.9199492568392196,
'test_mae': 124.71615341677166,
'overfit_gap_r2': 0.06362703477592757}

```

Random forest seems to be performing even better.

```

In [36]: param_grid_xgb = {
        "model__n_estimators": [400, 600],
        "model__max_depth": [5, 7],
        "model__learning_rate": [0.01, 0.1],
        "model__subsample": [0.8, 1.0],
        "model__colsample_bytree": [0.8, 1.0],
        "model__min_child_weight": [1, 3],
        "model__gamma": [0, 0.1]
    }

metrics_xgb_3, bestmodel_xgb_3 = evaluate_model(
    "XGBoost - without 7 least important features.",
    XGBRegressor(
        random_state=42,
        objective="reg:squarederror"
    ),
    param_grid_xgb,
    X_train_3,
    y_train,
    X_test_3,
    y_test,
    feature_cols=features_3
)

metrics_xgb_3

```

```

Out[36]: {'model': 'XGBoost - without 7 least important features.',
'best_params': {'model__colsample_bytree': 1.0,
'model__gamma': 0.1,
'model__learning_rate': 0.1,
'model__max_depth': 7,
'model__min_child_weight': 1,
'model__n_estimators': 600,
'model__subsample': 0.8},
'cv_rmse': 218.82289186195982,
'train_rmse': 91.25260253268166,
'test_rmse': 177.2901258688504,
'train_r2': 0.9848732979614432,
'test_r2': 0.9398037341128603,
'test_mae': 107.31663482900917,
'overfit_gap_r2': 0.04506956384858285}

```

And XGBoost performs much more better.

Part 3: Model comparison and picking the most suitable model

Now that we trained all the models, it would be better if we compare them in details before picking the final model, even though we noticed XGBoost performed the best.

```

In [37]: results = [
        metrics_linear,
        metrics_rf,
        metrics_rf_2,
        metrics_xgb,
        metrics_knn,
    ]

```

```

metrics_svr,
metrics_nn,
metrics_nn_2,
metrics_rf_3,
metrics_xgb_3
]

results_df = pd.DataFrame(results)

results_df

```

Out[37]:

	model	best_params	cv_rmse	train_rmse	test_rmse	train_r2	test_r2	test_mae	overfit_gap_r2
0	linear_regression	{}	432.648392	428.587877	408.987703	0.666317	0.679653	289.313141	-0.013335
1	Random Forest	{'model__max_depth': 30, 'model__max_features': ...}	234.580624	108.987300	209.193880	0.978422	0.916190	125.334425	0.062233
2	Random Forest - no bathtub/terrasse	{'model__max_depth': 40, 'model__max_features': ...}	234.220799	95.454310	206.311562	0.983448	0.918483	123.627334	0.064965
3	XGBoost	{'model__colsample_bytree': 0.8, 'model__gamma': ...}	213.525741	115.660841	184.139173	0.975699	0.935063	111.740268	0.040636
4	knn	{'model__n_neighbors': 3, 'model__weights': 'd...	295.784776	46.105525	265.524793	0.996138	0.864976	153.741122	0.131162
5	SVR	{'model__regressor__C': 0.1, 'model__regressor': ...}	437.011182	434.312876	408.331666	0.657343	0.680680	286.559764	-0.023336
6	Neural Network	{'epochs': 300, 'batch_size': 64}	NaN	178.197473	221.425479	0.942316	0.906102	142.047423	0.036214
7	Neural Network	{'epochs': 300, 'batch_size': 64}	NaN	180.641010	222.547166	0.940723	0.905148	144.113651	0.035575
8	Random Forest - without 7 least important feat...	{'model__max_depth': 40, 'model__max_features': ...}	233.669882	95.084285	204.447807	0.983576	0.919949	124.716153	0.063627
9	XGBoost - without 7 least important features.	{'model__colsample_bytree': 1.0, 'model__gamma': ...}	218.822892	91.252603	177.290126	0.984873	0.939804	107.316635	0.045070

In [38]:

```

results_df = pd.DataFrame(results)[
    [
        "model",
        "cv_rmse",
        "train_rmse",
        "test_rmse",
        "train_r2",
        "test_r2",
        "test_mae",
        "overfit_gap_r2",
    ]
]

results_df

```

Out[38]:

	model	cv_rmse	train_rmse	test_rmse	train_r2	test_r2	test_mae	overfit_gap_r2
0	linear_regression	432.648392	428.587877	408.987703	0.666317	0.679653	289.313141	-0.013335
1	Random Forest	234.580624	108.987300	209.193880	0.978422	0.916190	125.334425	0.062233
2	Random Forest - no bathtub/terrasse	234.220799	95.454310	206.311562	0.983448	0.918483	123.627334	0.064965
3	XGBoost	213.525741	115.660841	184.139173	0.975699	0.935063	111.740268	0.040636
4	knn	295.784776	46.105525	265.524793	0.996138	0.864976	153.741122	0.131162
5	SVR	437.011182	434.312876	408.331666	0.657343	0.680680	286.559764	-0.023336
6	Neural Network	NaN	178.197473	221.425479	0.942316	0.906102	142.047423	0.036214
7	Neural Network	NaN	180.641010	222.547166	0.940723	0.905148	144.113651	0.035575
8	Random Forest - without 7 least important feat...	233.669882	95.084285	204.447807	0.983576	0.919949	124.716153	0.063627
9	XGBoost - without 7 least important features.	218.822892	91.252603	177.290126	0.984873	0.939804	107.316635	0.045070

In [39]:

```

results_df.loc[7, "model"] = "Neural Network - no bathtub/terrasse"

```

In [40]:

```

results_df = results_df.sort_values(
    "test_r2",
    ascending=False
)

```

```
)  
results_df
```

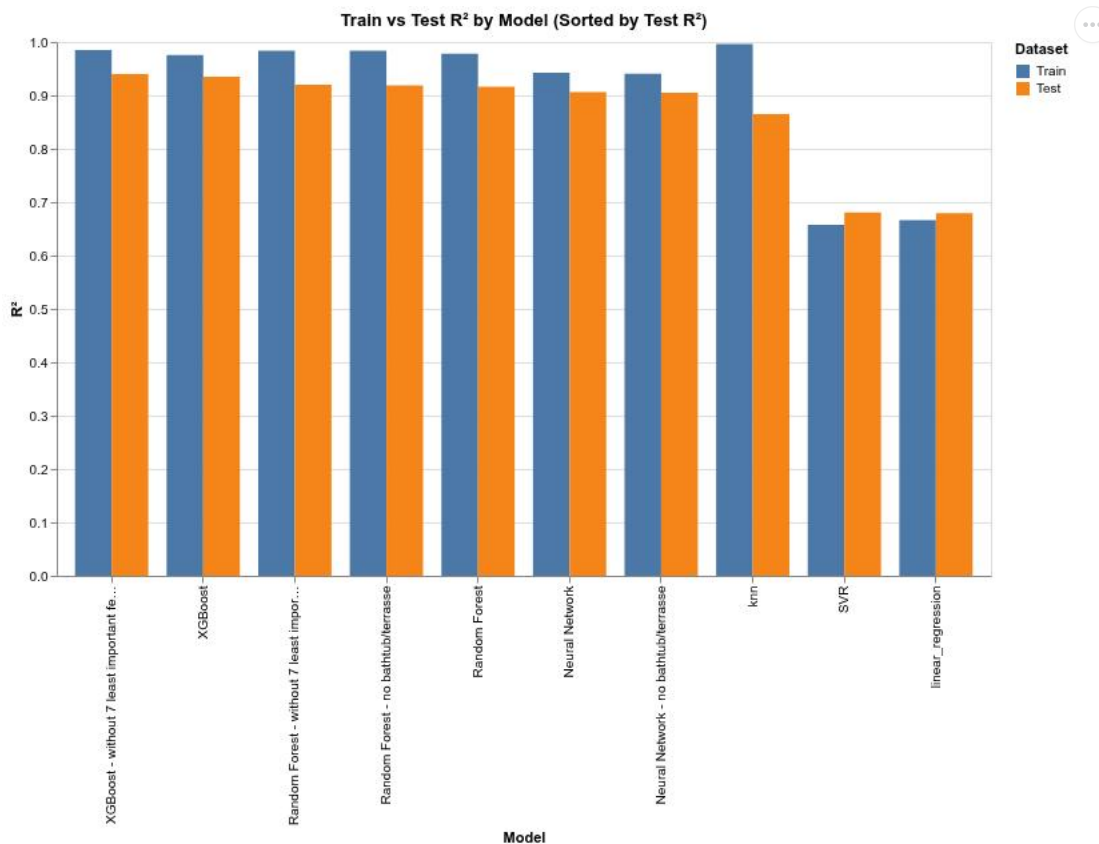
Out[40]:

	model	cv_rmse	train_rmse	test_rmse	train_r2	test_r2	test_mae	overfit_gap_r2
9	XGBoost - without 7 least important features.	218.822892	91.252603	177.290126	0.984873	0.939804	107.316635	0.045070
3	XGBoost	213.525741	115.660841	184.139173	0.975699	0.935063	111.740268	0.040636
8	Random Forest - without 7 least important feat...	233.669882	95.084285	204.447807	0.983576	0.919949	124.716153	0.063627
2	Random Forest - no bathtub/terrasse	234.220799	95.454310	206.311562	0.983448	0.918483	123.627334	0.064965
1	Random Forest	234.580624	108.987300	209.193880	0.978422	0.916190	125.334425	0.062233
6	Neural Network	NaN	178.197473	221.425479	0.942316	0.906102	142.047423	0.036214
7	Neural Network - no bathtub/terrasse	NaN	180.641010	222.547166	0.940723	0.905148	144.113651	0.035575
4	knn	295.784776	46.105525	265.524793	0.996138	0.864976	153.741122	0.131162
5	SVR	437.011182	434.312876	408.331666	0.657343	0.680680	286.559764	-0.023336
0	linear_regression	432.648392	428.587877	408.987703	0.666317	0.679653	289.313141	-0.013335

In [41]:

```
# ordered by test_r2 descending  
model_order = results_df.sort_values(by="test_r2", ascending=False)["model"].tolist()  
  
# reshape data to long format  
results_long = results_df.melt(  
    id_vars="model",  
    value_vars=["train_r2", "test_r2"],  
    var_name="dataset",  
    value_name="r2"  
)  
  
results_long["dataset"] = results_long["dataset"].replace({  
    "train_r2": "Train",  
    "test_r2": "Test"  
})  
  
dataset_order = ["Train", "Test"]  
  
chart = (  
    alt.Chart(results_long)  
    .mark_bar()  
    .encode(  
        x=alt.X(  
            "model:N",  
            title="Model",  
            sort=model_order # Explicitly set x-axis sort order by test_r2  
        ),  
        xOffset=alt.XOffset(  
            "dataset:N",  
            title="Dataset",  
            sort=dataset_order  
        ),  
        y=alt.Y(  
            "r2:Q",  
            title="R²",  
            scale=alt.Scale(domain=[0, 1])  
        ),  
        color=alt.Color(  
            "dataset:N",  
            title="Dataset",  
            sort=dataset_order  
        ),  
        tooltip=[  
            alt.Tooltip("model:N", title="Model"),  
            alt.Tooltip("dataset:N", title="Dataset"),  
            alt.Tooltip("r2:Q", title="R²", format=".3f")  
        ]  
    )  
    .properties(  
        width=700,  
        height=400,  
        title="Train vs Test R² by Model (Sorted by Test R²)"  
    )  
)  
chart
```

Out[41]:



We can notice that the differences among the tree models and NNs are minor. Also, KNN seems to have the highest training accuracy out of all of them, that is clearly a sign of overfitting. SVR and linear regression perform much worse compared to the rest.

```
In [43]: import plotly.graph_objects as go
from plotly.subplots import make_subplots

def _model_family(model_name):
    return model_name.split(" - ", 1)[0]

def build_radar(selected_results, all_results, title):
    metrics = ["test_r2", "cv_rmse", "test_rmse", "test_mae"]
    labels = ["Test R²", "CV RMSE", "Test RMSE", "Test MAE"]
    higher_is_better = [True, False, False, False]

    # Make a normalized copy of all results
    normalized = all_results.copy()

    # Normalize each metric
    for metric, higher in zip(metrics, higher_is_better):
        lo = normalized[metric].min()
        hi = normalized[metric].max()

        normalized[f"_radar_{metric}"] = (
            1.0 if hi == lo else
            (normalized[metric] - lo) / (hi - lo)
        )

        # Invert metrics where lower is better
        if not higher:
            normalized[f"_radar_{metric}"] = (
                1 - normalized[f"_radar_{metric}"]
            )

    # Get the normalized versions of the selected models
    selected_normalized = normalized.loc[selected_results.index]

    fig = go.Figure()

    for _, row in selected_normalized.iterrows():
        values = [
            float(row[f"_radar_{metric}"])
            for metric in metrics
        ]
```

```

    ]

    values.append(values[0])

    fig.add_trace(go.Scatterpolar(
        r=values,
        theta=labels + [labels[0]],
        fill="toself",
        name=row["model"],
    ))

    fig.update_layout(
        title=title,
        polar=dict(
            radialaxis=dict(
                visible=True,
                range=[0, 1]
            )
        ),
        showlegend=True,
    )

    return fig

# Keep only the best run from each model family.
best_by_family = (
    results_df.assign(family=results_df["model"].map(_model_family))
    .sort_values("test_r2", ascending=False)
    .drop_duplicates("family")
    .drop(columns="family")
)

top_4_models = results_df.head(4).copy()
fig_top4 = build_radar(top_4_models, results_df, "Top 4 Model Runs")

top_4_families = best_by_family.head(4).copy()
fig_families4 = build_radar(
    top_4_families,
    results_df,
    "Top 4 Different Model Families",
)

combined_fig = make_subplots(
    rows=1,
    cols=2,
    specs=[{"type": "polar"}, {"type": "polar"}],
    subplot_titles=["Top 4 Model Runs", "Top 4 Different Model Families"],
)

for trace in fig_top4.data:
    combined_fig.add_trace(trace, row=1, col=1)

for trace in fig_families4.data:
    combined_fig.add_trace(trace, row=1, col=2)

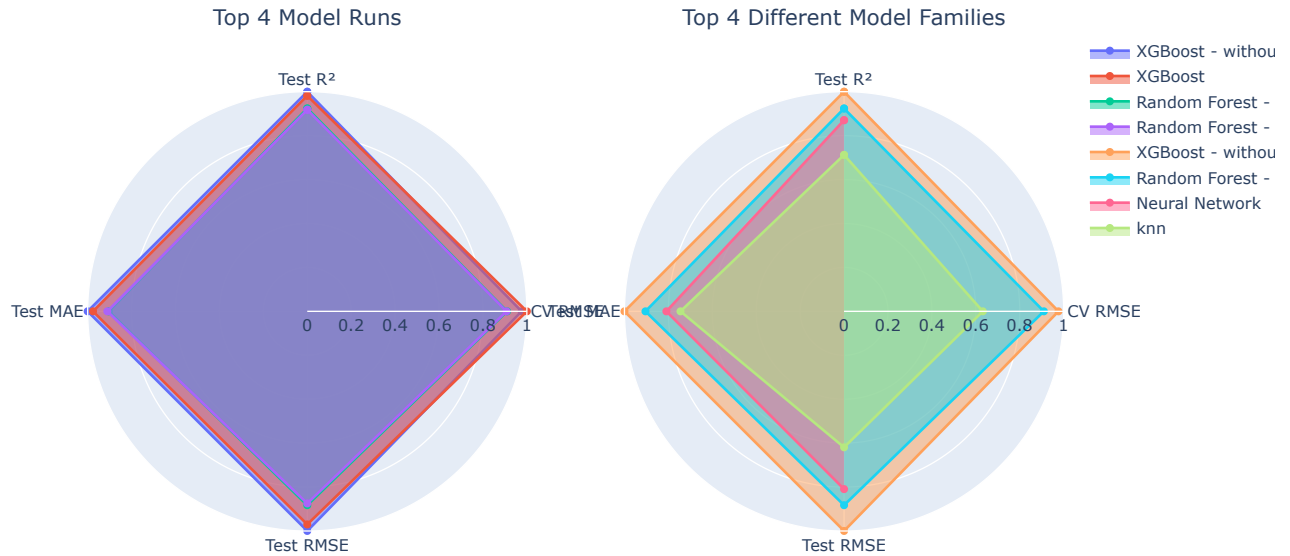
combined_fig.update_layout(
    width=1200,
    height=600,
    title="Model Comparison",
    showlegend=True,
)

combined_fig.update_polars(
    radialaxis=dict(
        visible=True,
        range=[0, 1],
        tickvals=[0, 0.2, 0.4, 0.6, 0.8, 1.0],
    ),
    angularaxis=dict(direction="clockwise"),
)

combined_fig.show()

```

Model Comparison



It is good to consider all types metrics, not just R^2 for the testing set. Radar charts are a good type of visualization to display and compare model performance for different kind of metrics. In the first radar chart we compare the 4 best performing models, 2 XGBoosts and 2 Random Forests. XGBoost without the 7 features seems to perform better in all 4 metrics. To get a view of how different model families perform, in the 2nd graph, we compare XGBoost, Random Forest, Neural Network and knn. Again, XGBoost performs the best in all metrics except in Generalization Gap. Neural Network seems to have the smallest Generalization Gap. This could suggest that the Neural Network model could deal better with overfitting. Also it is important to note that the metrics here are normalized and their directions are adjusted to show that the higher the value from 0 to 1, the better the metric is. The original values are still displayed on hover.

One very important thing about our project is that we don't just want to estimate a single point, but rather also suggest the user the interval of prices where their true could lie. For that we need some kind of uncertainty or confidence from the model and the standard XGBoost doesn't provide that, for this reason we need to train a variant of XGBoost called 'XGBoost Quantile Regression'. There is always a trade-off between things and therefore this model might cost us some prediction accuracy.

```
In [44]: from sklearn.pipeline import Pipeline
from xgboost import XGBRegressor

quantile_pipeline = Pipeline([
    ("preprocessor", build_preprocessor(features_2)),
    ("model", XGBRegressor(
        objective="reg:quantileerror",
        quantile_alpha=0.5,
        n_estimators=600,
        max_depth=7,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
    ))
])

quantile_pipeline.fit(X_train_2, y_train)

y_pred_q50 = quantile_pipeline.predict(X_test_2)

y_pred_q50 = np.asarray(y_pred_q50).ravel()
```

```
In [45]: from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

r2_q50 = r2_score(y_test, y_pred_q50)
rmse_q50 = np.sqrt(mean_squared_error(y_test, y_pred_q50))
mae_q50 = mean_absolute_error(y_test, y_pred_q50)
```

```

print("Quantile XGBoost (50%)")
print("Test R²:", r2_q50)
print("Test RMSE:", rmse_q50)
print("Test MAE:", mae_q50)

```

```

Quantile XGBoost (50%)
Test R²: 0.8869729524893049
Test RMSE: 242.9354895972518
Test MAE: 144.59292423700967

```

It seems like the model seems to be performing much poorer than the original XGBoost and even poorer than RandomForest. There is no point in keeping XGBoost, when RandomForest itself natively provides a way to estimate some model uncertainty and intervals of predicted values, therefore we will choose Random Forest as our final model.

Let's try to tune and select an optimal number of estimators and depth without overfitting the model.

```

In [49]: from sklearn.model_selection import cross_validate
from sklearn.metrics import make_scorer, mean_squared_error

depths = [2, 4, 6, 8, 10, 12, 15, 20, 25, 30, 35, 40, 50, 60, 80, 100]

depth_results = []

for depth in depths:
    rf_pipeline = build_pipeline(
        RandomForestRegressor(
            n_estimators=200,
            max_depth=depth,
            min_samples_split=2,
            min_samples_leaf=1,
            max_features="sqrt",
            random_state=42,
            n_jobs=-1
        ),
        feature_cols=features_2
    )

    scores = cross_validate(
        rf_pipeline,
        X_train_2,
        y_train,
        cv=make_cv(),
        scoring={
            "rmse": "neg_root_mean_squared_error",
            "r2": "r2"
        },
        return_train_score=True,
        n_jobs=-1
    )

    depth_results.append({
        "max_depth": depth,

        "train_rmse": -scores["train_rmse"].mean(),
        "val_rmse": -scores["test_rmse"].mean(),

        "train_r2": scores["train_r2"].mean(),
        "val_r2": scores["test_r2"].mean(),

        "rmse_gap": (
            -scores["test_rmse"].mean()
            - (-scores["train_rmse"].mean())
        ),

        "r2_gap": (
            scores["train_r2"].mean()
            - scores["test_r2"].mean()
        )
    })

depth_results = pd.DataFrame(depth_results)

depth_results

```


Out[49]:

	max_depth	train_rmse	val_rmse	train_r2	val_r2	rmse_gap	r2_gap
0	2	602.559103	603.280365	0.340414	0.338584	0.721262	0.001829
1	4	519.260157	522.486759	0.510145	0.503857	3.226602	0.006288
2	6	449.447032	457.978859	0.633010	0.618694	8.531827	0.014316
3	8	385.460040	403.481659	0.730034	0.703983	18.021619	0.026051
4	10	326.974167	355.356921	0.805746	0.770279	28.382754	0.035467
5	12	276.264325	316.646869	0.861333	0.817505	40.382544	0.043829
6	15	216.699265	277.863905	0.914683	0.859379	61.164640	0.055304
7	20	156.919143	247.667459	0.955258	0.888312	90.748316	0.066946
8	25	127.732088	237.557513	0.970354	0.897249	109.825426	0.073105
9	30	111.862744	234.791385	0.977264	0.899642	122.928641	0.077623
10	35	104.266079	234.447251	0.980248	0.899950	130.181172	0.080298
11	40	99.482416	234.742533	0.982017	0.899689	135.260117	0.082328
12	50	95.625829	233.898651	0.983385	0.900425	138.272822	0.082959
13	60	94.939370	234.369717	0.983622	0.899971	139.430347	0.083651
14	80	94.835297	234.105759	0.983658	0.900202	139.270463	0.083455
15	100	94.835297	234.105759	0.983658	0.900202	139.270463	0.083455

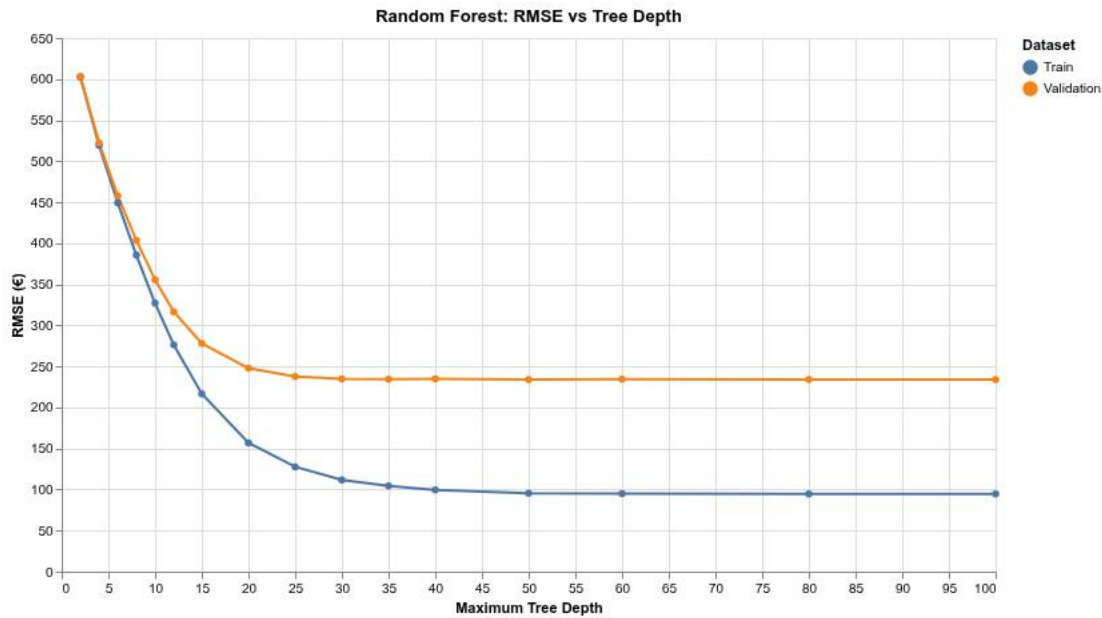
```
In [50]: depth_long = depth_results.melt(
    id_vars="max_depth",
    value_vars=["train_rmse", "val_rmse"],
    var_name="dataset",
    value_name="rmse"
)

depth_long["dataset"] = depth_long["dataset"].replace({
    "train_rmse": "Train",
    "val_rmse": "Validation"
})

chart_rmse = (
    alt.Chart(depth_long)
    .mark_line(point=True)
    .encode(
        x=alt.X(
            "max_depth:Q",
            title="Maximum Tree Depth"
        ),
        y=alt.Y(
            "rmse:Q",
            title="RMSE (€)"
        ),
        color=alt.Color(
            "dataset:N",
            title="Dataset"
        ),
        tooltip=[
            alt.Tooltip("max_depth:Q", title="Max Depth"),
            alt.Tooltip("dataset:N", title="Dataset"),
            alt.Tooltip("rmse:Q", title="RMSE", format=".2f")
        ]
    )
    .properties(
        width=700,
        height=400,
        title="Random Forest: RMSE vs Tree Depth"
    )
)

chart_rmse
```

Out[50]:



```
In [51]: n_estimators_values = [
          5, 10, 25, 50, 100, 150, 200, 300
        ]

n_estimators_results = []

for n in n_estimators_values:

    rf_pipeline = build_pipeline(
        RandomForestRegressor(
            n_estimators=n,
            max_depth=40,      # fixed
            min_samples_split=2, # fixed
            min_samples_leaf=1, # fixed
            max_features="sqrt", # fixed
            random_state=42,
            n_jobs=-1
        ),
        feature_cols=features_2
    )

    scores = cross_validate(
        rf_pipeline,
        X_train_2,
        y_train,
        cv=make_cv(),
        scoring={
            "rmse": "neg_root_mean_squared_error",
            "r2": "r2"
        },
        return_train_score=True,
        n_jobs=-1
    )

    n_estimators_results.append({
        "n_estimators": n,
        "train_rmse": -scores["train_rmse"].mean(),
        "val_rmse": -scores["test_rmse"].mean(),
        "train_r2": scores["train_r2"].mean(),
        "val_r2": scores["test_r2"].mean(),
        "rmse_gap": (
            -scores["test_rmse"].mean()
            - (-scores["train_rmse"].mean())
        ),
        "r2_gap": (
            scores["train_r2"].mean()
            - scores["test_r2"].mean()
        )
    })

n_estimators_results = pd.DataFrame(n_estimators_results)

n_estimators_results
```

```
Out[51]:
```

	n_estimators	train_rmse	val_rmse	train_r2	val_r2	rmse_gap	r2_gap
0	5	142.983959	277.194073	0.962854	0.859905	134.210114	0.102949
1	10	120.669104	256.572209	0.973541	0.880091	135.903104	0.093450
2	25	107.580759	242.275466	0.978969	0.893114	134.694707	0.085855
3	50	103.290421	238.066872	0.980611	0.896829	134.776451	0.083782
4	100	100.566870	235.106690	0.981622	0.899367	134.539820	0.082255
5	150	99.836493	235.083602	0.981887	0.899409	135.247109	0.082479
6	200	99.482416	234.742533	0.982017	0.899689	135.260117	0.082328
7	300	99.110327	234.330349	0.982150	0.900031	135.220022	0.082119

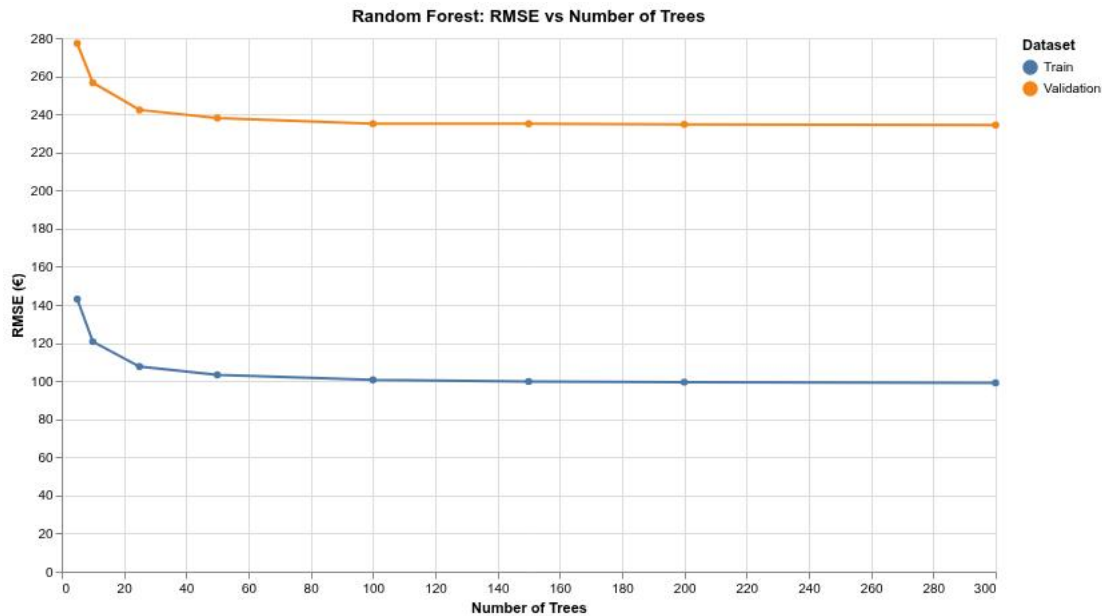
```
In [52]: n_estimators_long = n_estimators_results.melt(
    id_vars="n_estimators",
    value_vars=["train_rmse", "val_rmse"],
    var_name="dataset",
    value_name="rmse"
)

n_estimators_long["dataset"] = n_estimators_long["dataset"].replace({
    "train_rmse": "Train",
    "val_rmse": "Validation"
})

chart_n_estimators = (
    alt.Chart(n_estimators_long)
    .mark_line(point=True)
    .encode(
        x=alt.X(
            "n_estimators:Q",
            title="Number of Trees"
        ),
        y=alt.Y(
            "rmse:Q",
            title="RMSE (€)"
        ),
        color=alt.Color(
            "dataset:N",
            title="Dataset"
        ),
        tooltip=[
            alt.Tooltip(
                "n_estimators:Q",
                title="Number of Trees"
            ),
            alt.Tooltip(
                "dataset:N",
                title="Dataset"
            ),
            alt.Tooltip(
                "rmse:Q",
                title="RMSE",
                format=".2f"
            )
        ]
    )
    .properties(
        width=700,
        height=400,
        title="Random Forest: RMSE vs Number of Trees"
    )
)

chart_n_estimators
```

Out[52]:



The optimal spot for this kind of graphs is the area around the 'elbow', the area where the line starts to flatten. Looking at the graphs, a number of estimators of 200 and a depth of 40, like we originally had seems a reasonable pick.

```
In [53]: metrics_final, final_model = evaluate_model(
    "Random Forest - Final",
    RandomForestRegressor(
        n_estimators=200,
        max_depth=40,
        min_samples_split=2,
        min_samples_leaf=1,
        max_features="sqrt",
        random_state=42,
        n_jobs=-1
    ),
    {},
    X_train_2,
    y_train,
    X_test_2,
    y_test,
    feature_cols=features_2
)
metrics_final
```

```
Out[53]: {'model': 'Random Forest - Final',
  'best_params': {},
  'cv_rmse': 234.8995180606382,
  'train_rmse': 96.11548891108951,
  'test_rmse': 208.06816963823266,
  'train_r2': 0.9832181245969392,
  'test_r2': 0.9170890772584996,
  'test_mae': 124.45708254768779,
  'overfit_gap_r2': 0.0661290473384396}
```

Part 4: Saving the model

```
In [54]: save_bundle(
    final_model,
    "final_model.pkl"
)
```

```
Out[54]: WindowsPath('final_model.pkl')
```